



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Aprendizaje Automático Probabilístico

Redes neuronales para secuencias

Alfons Juan

DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

15.1	Introducción	1
15.2	Redes neuronales recurrentes	2
15.2.1	Vec2Seq: generación de secuencias	3
15.2.1.1	Modelos	5
15.2.1.2	Aplicaciones	9
15.2.2	Seq2Vec: clasificación de secuencias	12
15.2.3	Seq2Seq: traducción de secuencias	16
15.2.3.1	Caso alineado	17
15.2.3.2	Caso no alineado	20
15.2.4	Aprendizaje forzado	24
15.2.5	Backpropagation through time	25
15.2.6	Gradientes que desaparecen y explotan	29
15.2.7	Gating y long term memory	30
15.2.7.1	Gated recurrent units (GRU)	31
15.2.7.2	Long short term memory (LSTM)	34
15.2.8	Beam search	37
15.3	CNNs unidimensionales	40

15.3.1	CNNs 1d para clasificación de secuencias	41
15.3.2	CNNs 1d causales para generar secuencias	44
15.4	Atención	46
15.4.1	Atención: búsqueda suave en diccionario	47
15.4.2	Regresión kernel: atención no paramétrica	50
15.4.3	Atención paramétrica	52
15.4.4	Seq2Seq con atención	54
15.4.5	Seq2Vec con atención	57
15.4.6	Seq+Seq2Vec con atención	58
15.4.7	Atención blanda y dura	62
15.5	Transformers	65
15.5.1	Auto-atención	66
15.5.2	Atención multi-cabecal	68
15.5.3	Codificación posicional	70
15.5.4	Todo junto	76
15.5.5	Comparación de transformers, CNNs y RNNs	79
15.5.6	Transformers para imágenes	81
15.5.7	Otras variantes transformer	82
15.6	Transformers eficientes	83

15.6.1	Patrones de atención localizados y fijos	84
15.6.2	Patrones de atención dispersos y aprendibles	85
15.6.3	Métodos recurrentes y memorísticos	86
15.6.4	Métodos basados en kernels y de bajo rango	87
15.7	Modelos de lenguaje	92
15.7.1	ELMo	94
15.7.2	BERT	97
15.7.2.1	Tarea del modelo de lenguaje enmascarado	98
15.7.2.2	Tarea de predicción de la siguiente frase	101
15.7.2.3	Fine-tuning BERT para aplicaciones NLP	103
15.7.3	GPT	108
15.7.3.1	Aplicaciones de GPT	110
15.7.4	T5	111
15.7.5	Discusión	113

15.1. Introducció

► *Tres casos:*

- ▷ La entrada es una secuencia
- ▷ La salida es una secuencia
- ▷ Tanto entrada como salida son secuencias

► *Aplicaciones:*

- ▷ Reconocimiento automático del habla
- ▷ Traducción automática
- ▷ Clasificación de texto
- ▷ Etiquetado de imágenes
- ▷ etc.

15.2. Redes neuronales recurrentes

► *Recurrent neural network (RNN):*

- ▷ Transforma de un espacio de secuencias de entrada a un espacio de secuencias de salida en función de su estado
- ▷ La de salida y_t no solo depende de la entrada x_t , sino también del estado oculto del sistema, h_t , el cual se actualiza a lo largo del tiempo, a medida que la secuencia de entrada se procesa
- ▷ Usos: generación, clasificación y traducción de secuencias

15.2.1. Vec2Seq: generación de secuencias

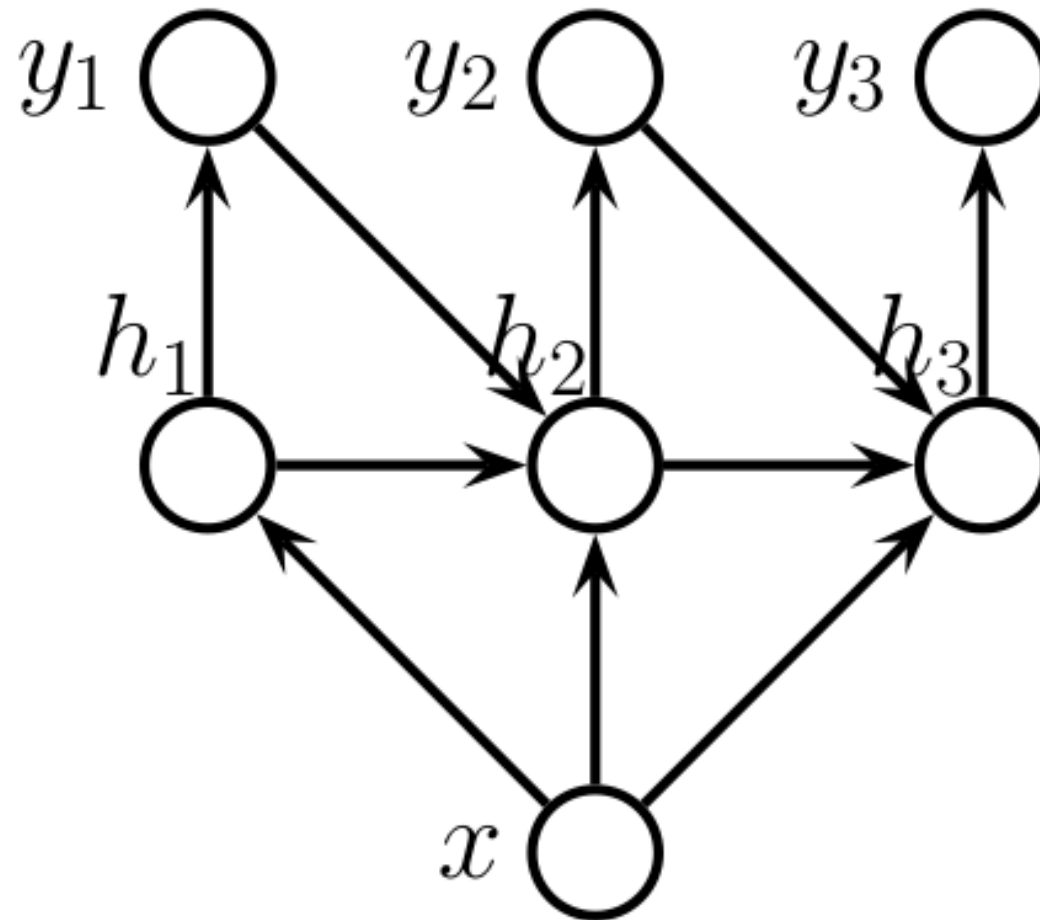
- ▶ Se trata de aprender funciones de la forma

$$f_{\Theta} : \mathbb{R}^D \rightarrow \mathbb{R}^{N_{\infty} C} \quad (1)$$

donde

- ▷ D es la dimensión del vector de entrada
- ▷ la salida es una secuencia de vectores C -dimensionales de longitud arbitraria

- ▶ La RNN es un modelo generativo condicional, $p(\mathbf{y}_{1:T} \mid \mathbf{x})$
 - ▷ Se genera un token de salida por cada t
 - ▷ $\mathbf{y}_t$ se genera a partir de $\mathbf{h}_t$ y re-alimenta el modelo para obtener $\mathbf{h}_{t+1}$ (también dependiente de $\mathbf{x}$)



15.2.1.1. Modelos

► **Modelo:** si T es la longitud de salida (escogida dinámicamente)

$$p(\mathbf{y}_{1:T} \mid \mathbf{x}) = \sum_{\mathbf{h}_{1:T}} p(\mathbf{y}_{1:T}, \mathbf{h}_{1:T} \mid \mathbf{x}) \quad (2)$$

$$= \sum_{\mathbf{h}_{1:T}} \prod_{t=1}^T p(\mathbf{y}_t \mid \mathbf{h}_t) p(\mathbf{h}_t \mid \mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x}) \quad (3)$$

donde

▷ $\mathbf{h}_t$ es el estado oculto

▷ definimos como distribución del estado oculto inicial:

$$p(\mathbf{h}_1 \mid \mathbf{h}_0, \mathbf{y}_0, \mathbf{x}) \triangleq p(\mathbf{h}_1 \mid \mathbf{x}) \quad (4)$$

► *Modelo de salida:*

▷ Parámetros: pesos oculto-salida $\mathbf{W}_{hy}$ y sesgo $\mathbf{b}_y$

▷ Salida discreta:

$$p(\mathbf{y}_t \mid \mathbf{h}_t) = \mathcal{M}(\mathbf{y}_t \mid \mathcal{S}(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)) \quad (5)$$

▷ Salida continua:

$$p(\mathbf{y}_t \mid \mathbf{h}_t) = \mathcal{N}(\mathbf{y}_t \mid \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \sigma^2\mathbf{I}) \quad (6)$$

► *Modelo oculto:*

- Asumimos que el estado oculto se calcula con una función determinista dada, f

$$p(\mathbf{h}_t \mid \mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x}) = \mathbb{1}(\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x})) \quad (7)$$

- La f suele darse a partir de pesos oculto-oculto $\mathbf{W}_{hh}$, entrada-oculto $\mathbf{W}_{xh}$ y sesgo $\mathbf{b}_h$, como:

$$\mathbf{h}_t = \varphi(\mathbf{W}_{xh}[\mathbf{x}; \mathbf{y}_{t-1}] + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \quad (8)$$

- El modelo no tiene memoria acotada ya que $\mathbf{y}_t$ depende de $\mathbf{h}_t$, que a su vez depende de $\mathbf{y}_{t-1}$, que depende de $\mathbf{h}_{t-1}$, etc.

► **Estocasticidad:**

▷ Viene dada por el modelo de salida:

↳ Muestreamos $\tilde{\mathbf{y}}_t \sim p(\mathbf{y}_t \mid \mathbf{h}_t)$

↳ $\mathbf{h}_{t+1} = f(\mathbf{h}_t, \tilde{\mathbf{y}}_t, \mathbf{x})$, esto es, $\mathbf{h}_{t+1}$ se halla de manera determinista en función de la $\tilde{\mathbf{y}}_t$ muestreada anteriormente

↳ A partir de $\mathbf{h}_{t+1}$, muestreamos $\tilde{\mathbf{y}}_{t+1} \sim p(\mathbf{y}_{t+1} \mid \mathbf{h}_{t+1})$

↳ etc.

▷ **RNN variacional:** añade estocasticidad a la dinámica de $\mathbf{h}_t$

15.2.1.2. Aplicaciones

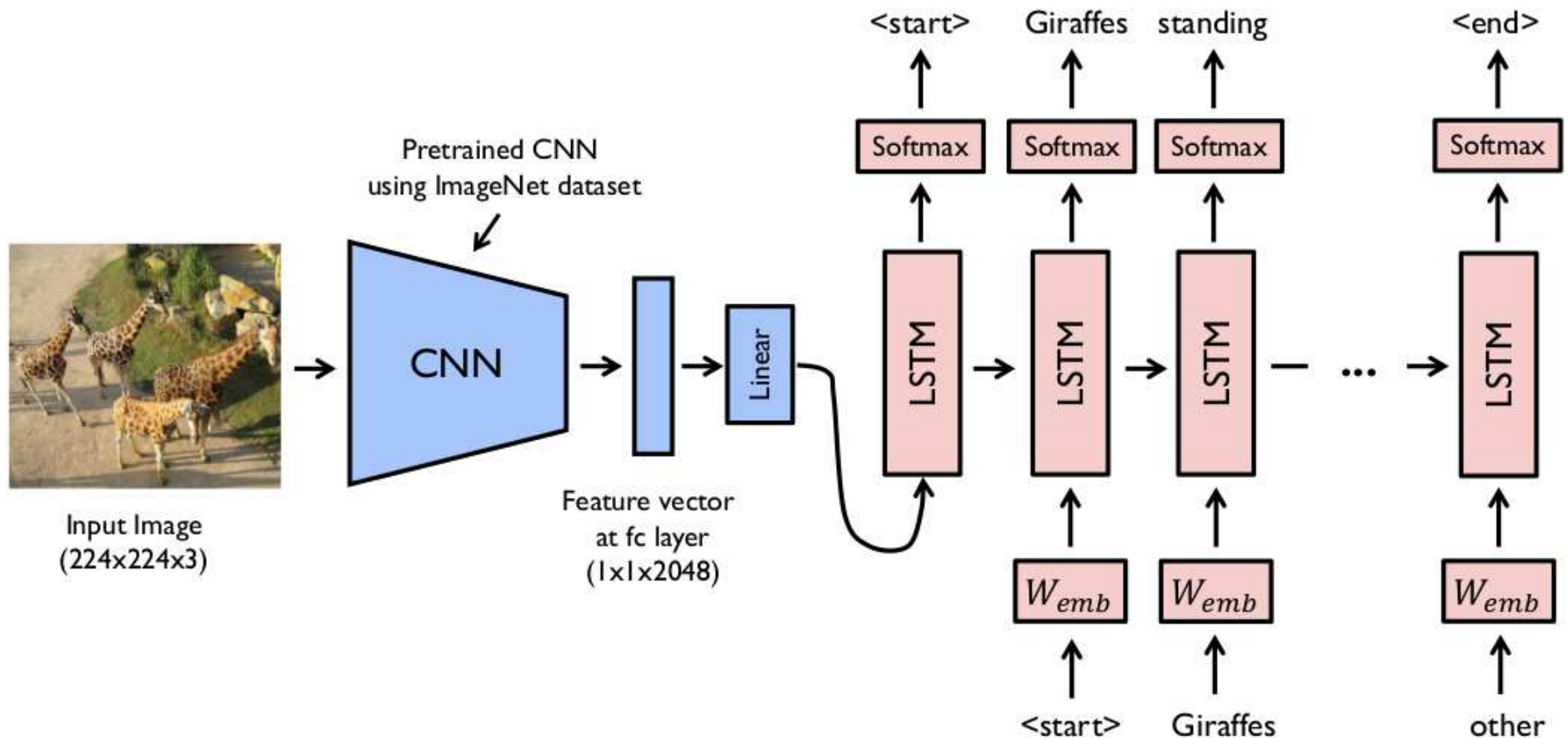
► *Modelado de lenguaje:*

- ▷ Generación de secuencias incondicional ($x = \emptyset$) con una RNN
- ▷ *Ejemplo:* secuencia generada a partir de “the” por una RNN de caracteres entrenada con *The Time Machine* de H. G. Wells

1 the githa some thong the time traveller held in his hand was a
2 glitteringmetallic framework scarcely larger than a small clock and
3 verydelicately made there was ivory in it and the latter than s bettyre
4 tat howhong s ie time thave ler simk you a dimensions le ghat dionthat
5 shall travel indifferently in any direction of space and timeas the
6 driver determinesfilby contented himself with laughterbut i have
7 experimental verification said the time travellerit would be remarkably
8 convenient for the histo

► Image captioning:

- Generación de secuencias condicionada por un vector x obtenido mediante embedding de la imagen de entrada con una CNN



- ▶ ***Otras aplicaciones:*** generación de secuencias de vectores de características reales
 - ▷ Trazos de caracteres manuscritos
 - ▷ Formas dibujadas a mano
 - ▷ Predicción de series temporales en general

15.2.2. Seq2Vec: clasificación de secuencias

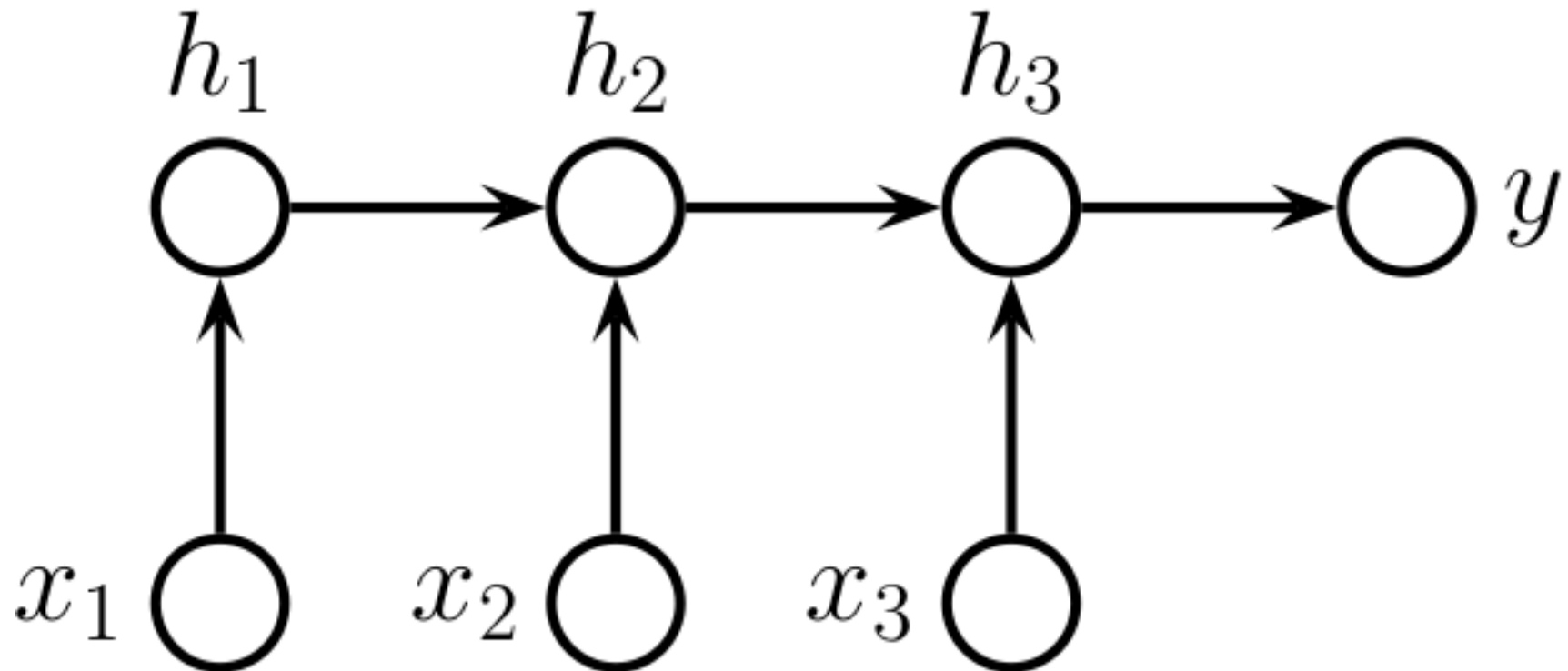
- ▶ Se trata de aprender funciones de la forma

$$f_{\Theta} : \mathbb{R}^{TD} \rightarrow \mathbb{R}^C \quad (9)$$

- ▷ **Clasificación de secuencias:** suponemos que la salida es una etiqueta de clase $y \in \{1, \dots, C\}$

- **Aproximación directa:** usar el estado final de la RNN como entrada al clasificador

$$p(y \mid \mathbf{x}_{1:T}) = \mathcal{M}(y \mid \mathcal{S}(\mathbf{W}\mathbf{h}_T)) \quad (10)$$



► *RNN bidireccional:*

- Se obtienen mejores resultados si los estados ocultos dependen no solo del contexto pasado, sino también del futuro
- El estado oculto h_t se compone del pasado $h_t^{\rightarrow}$ y el futuro $h_t^{\leftarrow}$:

$$h_t = [h_t^{\rightarrow}, h_t^{\leftarrow}]$$

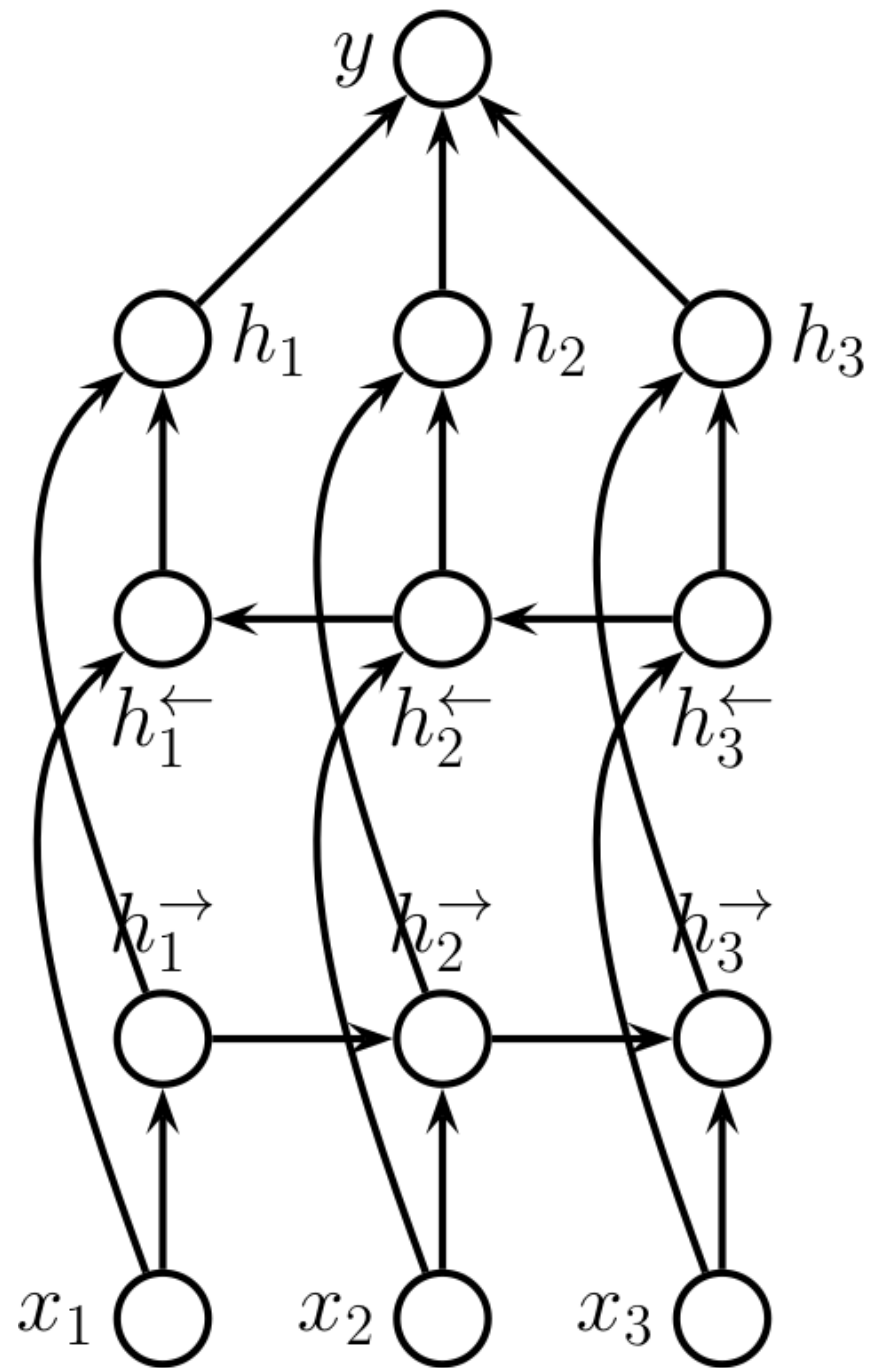
$$h_t^{\rightarrow} = \varphi(\mathbf{W}_{xh}^{\rightarrow} \mathbf{x}_t + \mathbf{W}_{hh}^{\rightarrow} h_{t-1}^{\rightarrow} + \mathbf{b}_h^{\rightarrow})$$

$$h_t^{\leftarrow} = \varphi(\mathbf{W}_{xh}^{\leftarrow} \mathbf{x}_t + \mathbf{W}_{hh}^{\leftarrow} h_{t+1}^{\leftarrow} + \mathbf{b}_h^{\leftarrow})$$

- El clasificador final promedia los estados ocultos:

$$p(y \mid \mathbf{x}_{1:T}) = \mathcal{M}(y \mid \mathbf{W}\mathcal{S}(\bar{\mathbf{h}}))$$

$$\bar{\mathbf{h}} = \frac{1}{T} \sum_{t=1}^T h_t$$



15.2.3. Seq2Seq: traducción de secuencias

- ▶ Se trata de aprender funciones de la forma

$$f_{\Theta} : \mathbb{R}^{TD} \rightarrow \mathbb{R}^{T'C} \quad (11)$$

- ▶ Consideramos dos casos separados:

- ▷ Alineado: $T' = T$

- ▷ No alineado: $T' \neq T$

15.2.3.1. Caso alineado

► **Dense sequence labeling:** se predice una etiqueta por posición

▷ **RNN (unidireccional):** resulta fácil modificarla

$$p(\mathbf{y}_{1:T} \mid \mathbf{x}_{1:T}) = \sum_{\mathbf{h}_{1:T}} \prod_{t=1}^T p(\mathbf{y}_t \mid \mathbf{h}_t) \mathbb{1}(\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)) \quad (12)$$

donde definimos $\mathbf{h}_1 = f(\mathbf{h}_0, \mathbf{x}_1) = f_0(\mathbf{x}_1)$ como estado inicial

▷ **RNN bidireccional:** en el modelo unidireccional, $\mathbf{y}_t$ solo depende de $\mathbf{h}_t$ y, a su vez, éste solo depende del pasado, $\mathbf{x}_{1:t}$; podemos obtener mejores resultados si permitimos que la salida también dependa del “futuro” de $\mathbf{x}$ mediante un modelo bidireccional

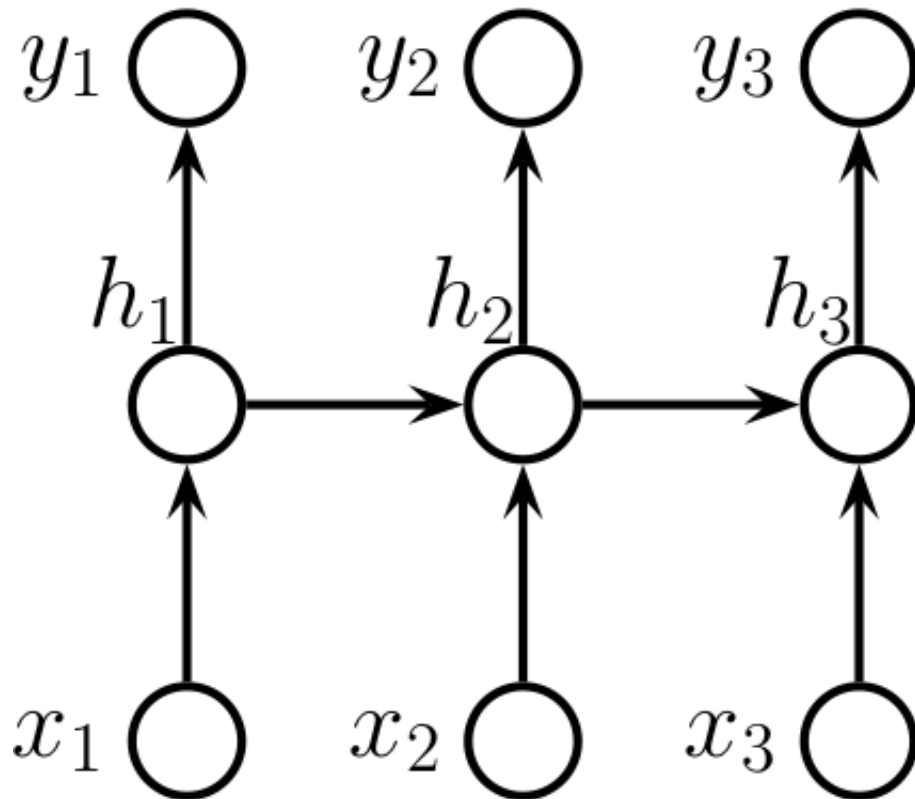
▷ **Múltiples cadenas ocultas:** unidad oculta en capa ℓ y t

$$\mathbf{h}_t^\ell = \varphi_\ell(\mathbf{W}_{xh}^\ell \mathbf{h}_t^{\ell-1} + \mathbf{W}_{hh}^\ell \mathbf{h}_{t-1}^\ell + \mathbf{h}_d^\ell) \quad (13)$$

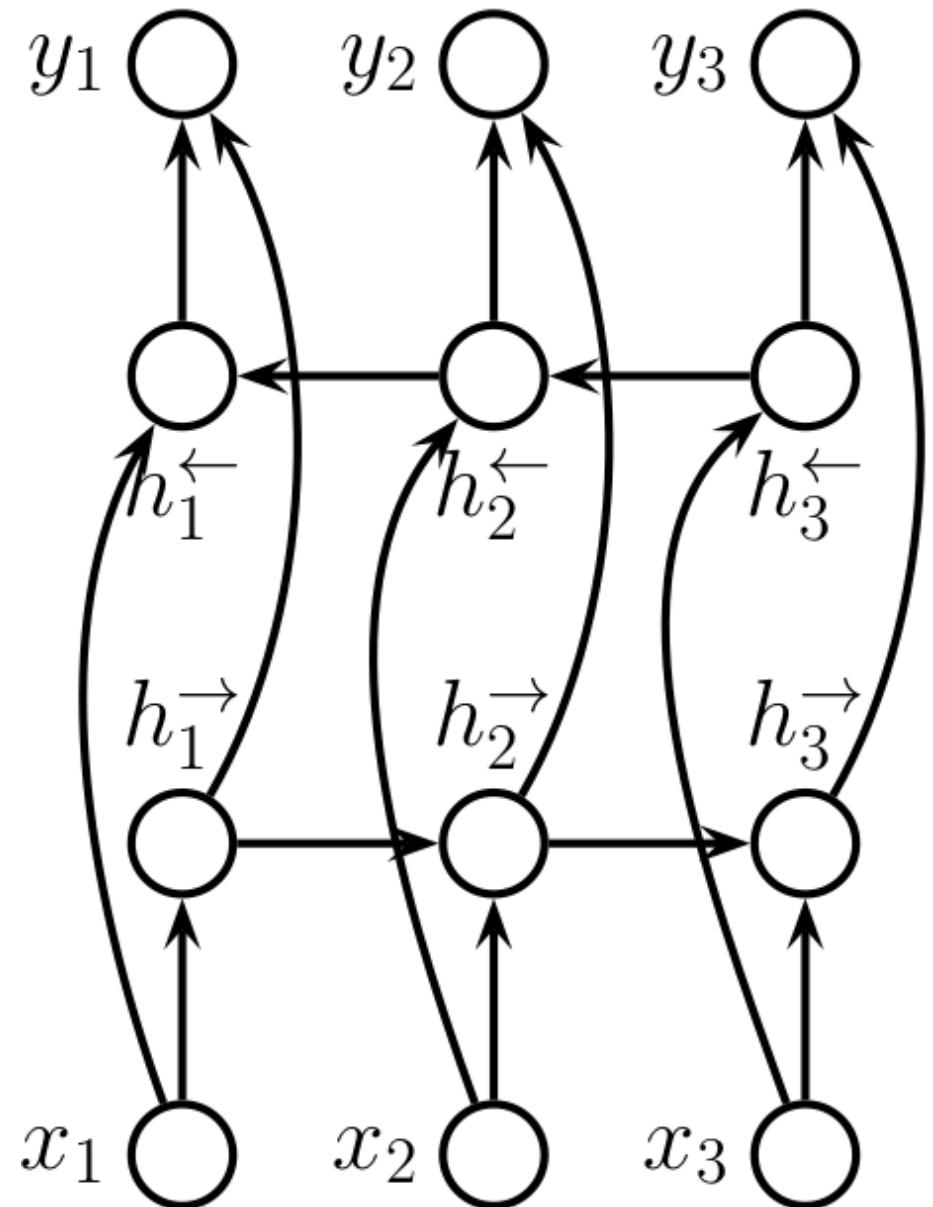
con salida

$$\mathbf{o}_t = \mathbf{W}_{ho} \mathbf{h}_t^L + \mathbf{b}_o \quad (14)$$

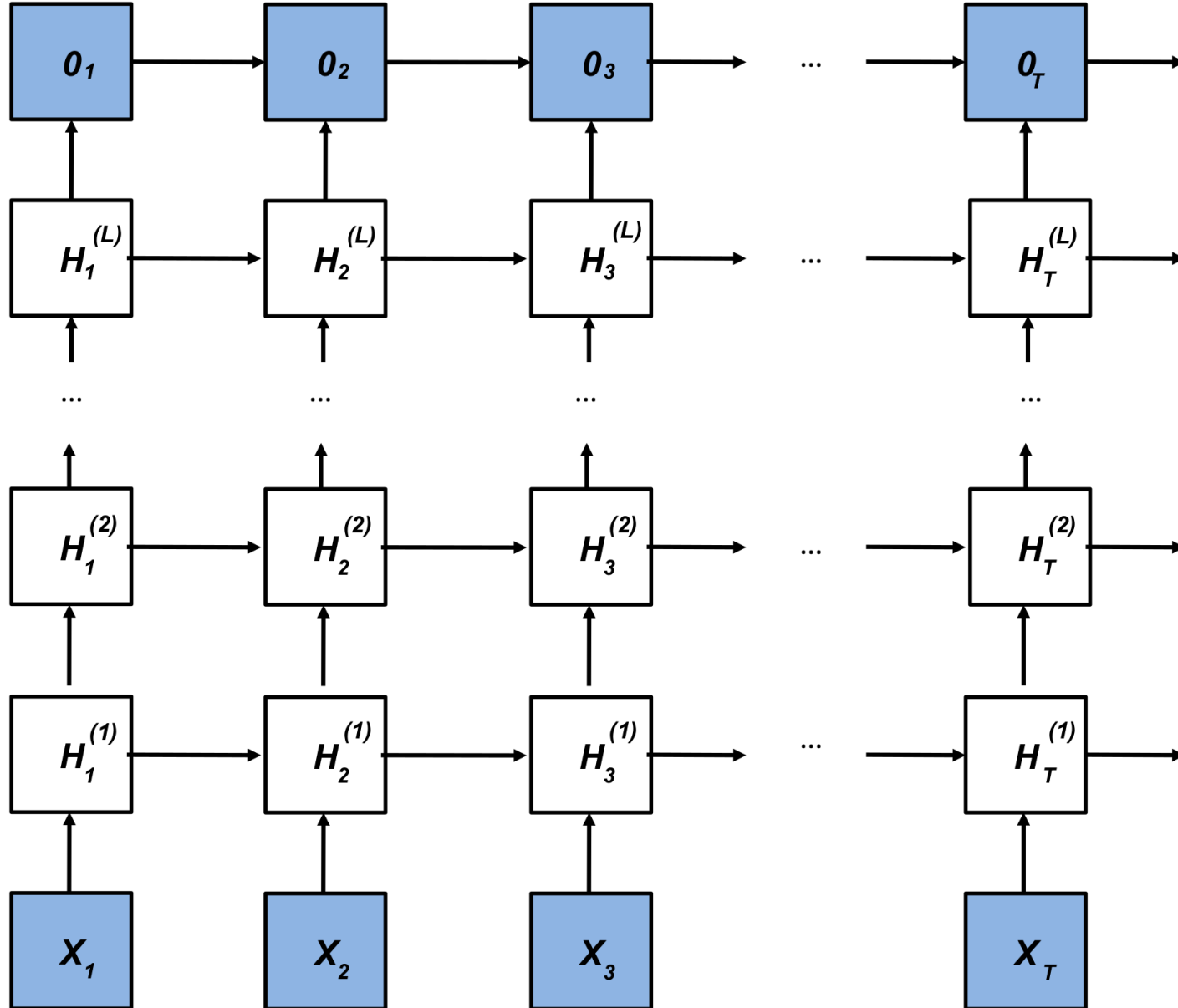
► *RNN unidireccional:*



► *RNN bidireccional:*



► *RNN con múltiples cadenas ocultas:*



15.2.3.2. Caso no alineado

► *Arquitectura encoder-decoder:*

- ▷ **Encoder:** codifica la secuencia de entrada para obtener un vector contextual usando el último estado de una RNN (o average pooling con una BRNN)

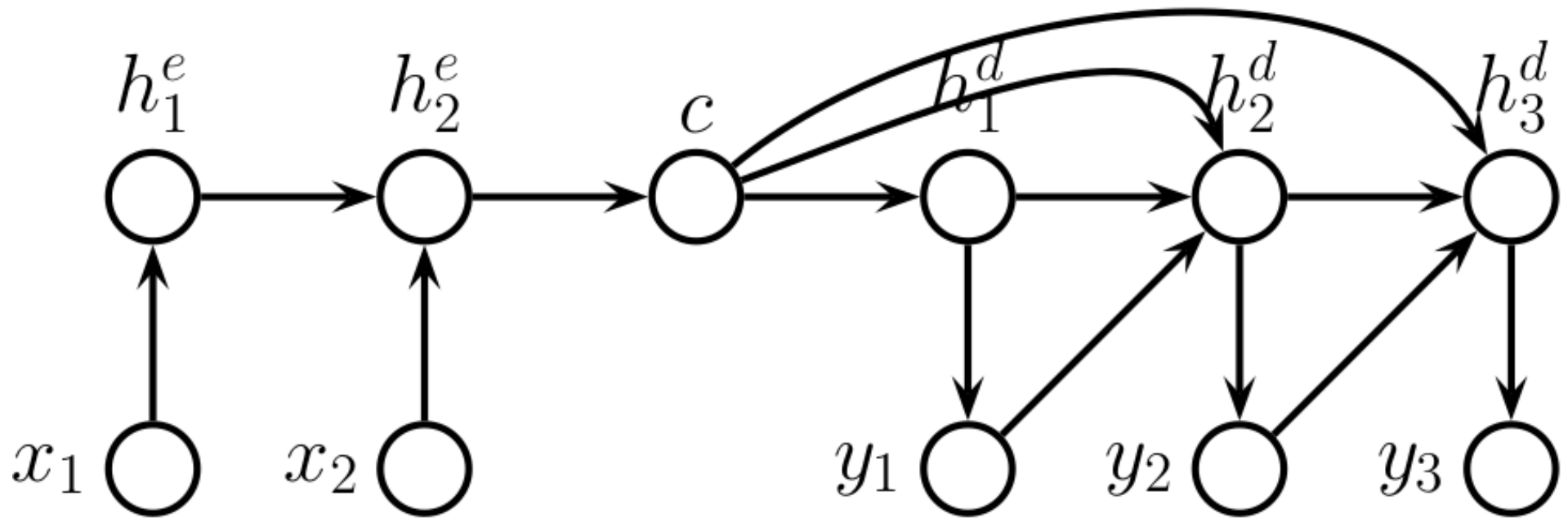
$$\mathbf{c} = f_e(\mathbf{x}_{1:T}) \quad (15)$$

- ▷ **Decoder:** genera la secuencia de salida mediante un decodificador RNN

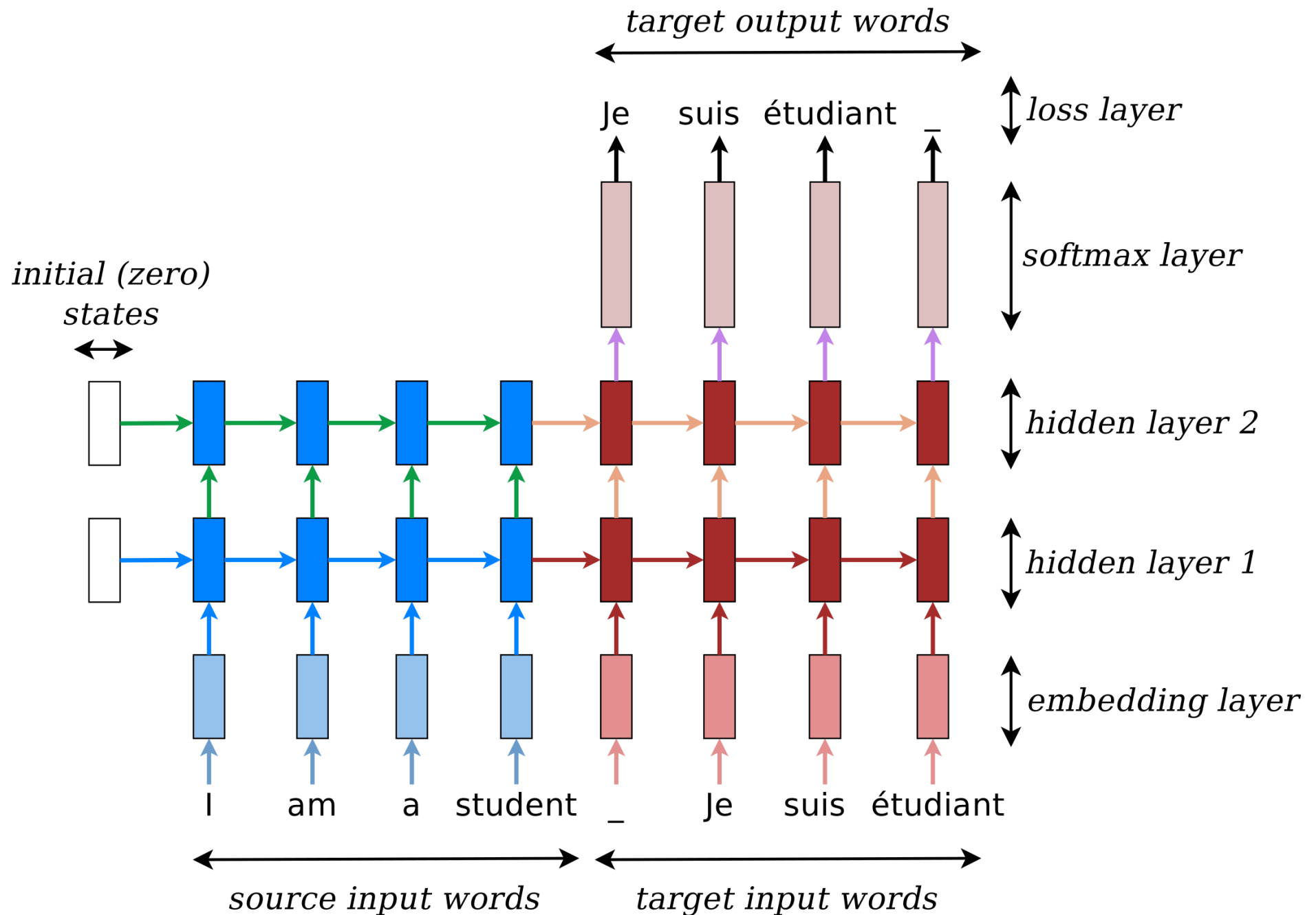
$$\mathbf{y}_{1:T'} = f_d(\mathbf{c}) \quad (16)$$

- ▷ **Neural machine translation (NMT):** aplicación destacada

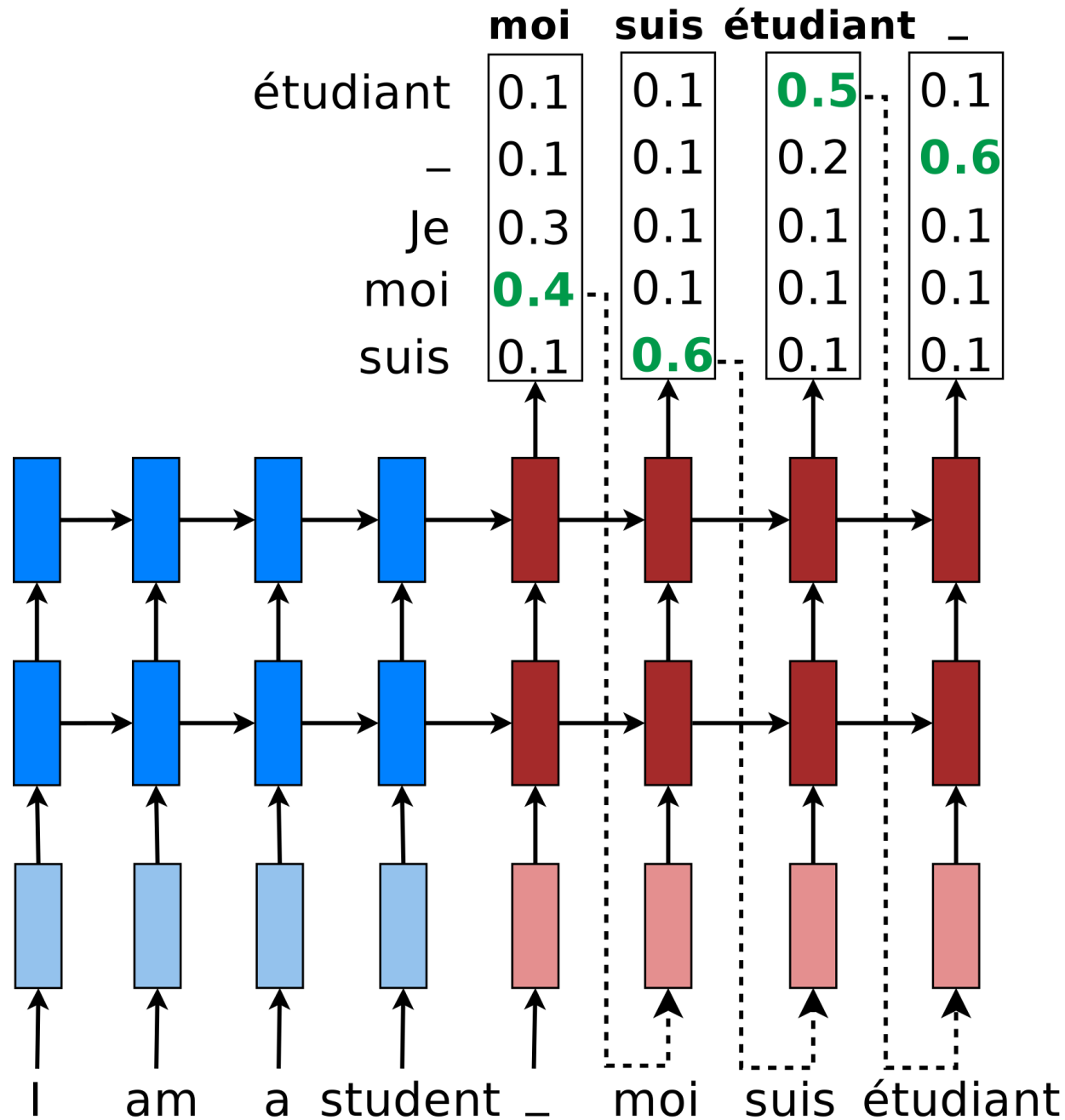
► *Representación de la arquitectura encoder-decoder:*



► *NMT: modelo seq2seq para traducir de Inglés a Francés*



► *NMT: decodificación voraz*



15.2.4. Aprendizaje forzado

- ▶ En entrenamiento de un modelo de lenguaje, la verosimilitud de una secuencia de palabras $w_1, w_2, \dots, w_T$ es

$$p(\mathbf{w}_{1:T}) = \prod_{t=1}^T p(w_t \mid \mathbf{w}_{1:t-1}) \quad (17)$$

- ▶ *Aprendizaje forzado con una RNN:*

- ▷ Fija la entrada a $x_t = w_{t-1}$ y la salida a $y_t = w_t$, tomando como etiquetas del pasado, $\mathbf{w}_{1:t-1}$, la *ground truth* en lugar de etiquetas generadas por el modelo
- ▷ En la práctica no da buenos resultados en inferencia, por lo que se proponen alternativas como el *scheduled sampling* o el uso de otros modelos (CNNs 1d y transformers)

15.2.5. Backpropagation through time

- MLE de una RNN: (con una única secuencia de entrenamiento)

$$\Theta^* = \arg \max_{\Theta} p(\mathbf{y}_{1:T} \mid \mathbf{x}_{1:T}, \Theta) \quad (18)$$

- *Backpropagation through time (BPTT)*: backprop aplicado al grafo computacional *desenrollado (unrolled)* en el tiempo
- *Modelo*: con logits de salida $\mathbf{o}_t$ y sin sesgos por simplicidad

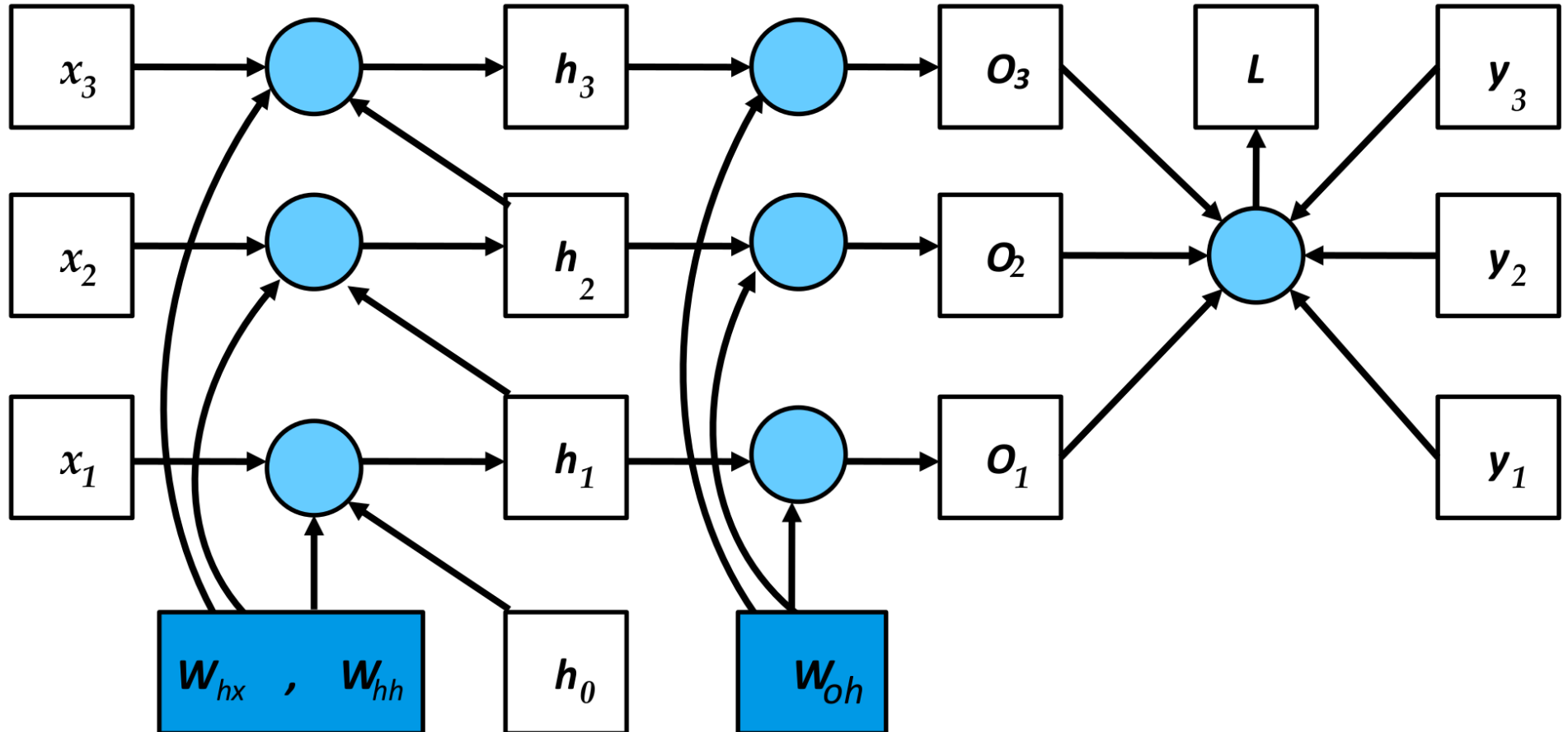
$$\mathbf{h}_t = \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} \quad (19)$$

$$\mathbf{o}_t = \mathbf{W}_{ho}\mathbf{h}_t \quad (20)$$

- *Pérdida*: con etiquetas de salida y_t

$$L = \frac{1}{T} \sum_{t=1}^T \ell(y_t, \mathbf{o}_t) \quad (21)$$

► *Representación gráfica del modelo:*



► **Derivadas a calcular:**

$$\frac{\partial L}{\partial \mathbf{W}_{hx}}, \quad \frac{\partial L}{\partial \mathbf{W}_{hh}}, \quad \frac{\partial L}{\partial \mathbf{W}_{ho}} \quad (22)$$

- La última es fácil de calcular ya que es local a cada t
- Las dos primeras dependen del estado oculto, por lo que requieren mirar atrás en el tiempo

► **Cálculo básico:** con $\mathbf{w}_h$ igual a $\mathbf{W}_{hh}$ y $\mathbf{W}_{hx}$ aplanadas y apiladas

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h) \quad (23)$$

$$\mathbf{o}_t = g(\mathbf{h}_t, \mathbf{w}_o) \quad (24)$$

- Por la regla de la cadena:

$$\frac{\partial L}{\partial \mathbf{w}_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial \ell(y_y, \mathbf{o}_t)}{\partial \mathbf{w}_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial \ell(y_y, \mathbf{o}_t)}{\partial \mathbf{o}_t} \frac{\partial g(\mathbf{h}_t, \mathbf{w}_o)}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} \quad (25)$$

▷ El último término puede expandirse como sigue:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} = \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h} + \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{w}_h} \quad (26)$$

▷ Si se sigue expandiendo recursivamente, se llega a:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} = \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h} + \sum_{i=1}^{t-1} \left[\prod_{j=i+1}^t \frac{\partial f(\mathbf{x}_j, \mathbf{h}_{j-1}, \mathbf{w}_h)}{\partial \mathbf{h}_{j-1}} \right] \frac{\partial f(\mathbf{x}_i, \mathbf{h}_{i-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h}$$

- ▶ **Coste temporal:** $O(T)$ por paso; $O(T^2)$ en total
- ▶ **Reducción del coste:** suma de K términos más recientes
- ▶ **Aplicación a secuencias largas:** con ventanas no solapadas sucesivas, manteniendo el estado oculto de la RNN en batches sucesivos durante el entrenamiento (sin barajar subsecuencias)

15.2.6. Gradientes que desaparecen y explotan

- ▶ **Vanishing and exploding gradients:** las activaciones en una RNN tienden a desaparecer o explotar...
 - ▷ en forward, ya que multiplicamos la matriz $\mathbf{W}_{hh}$ en cada t ,
 - ▷ y backward, ya que multiplicamos Jacobianas en cada t
- ▶ **Gradient clipping:** solución sencilla para los que explotan
- ▶ **Reservoir computing:** control del radio espectral λ del mapping forward, $\mathbf{W}_{hh}$, y del backward, dado por la Jacobiana $\mathbf{J}_{hh}$
 - ▷ **Echo state network (ESN):** inicializa aleatoriamente $\mathbf{W}_{hh}$ tal que $\lambda \approx 1$ y la mantiene fija, por lo que solo aprende $\mathbf{W}_{ho}$
 - ▷ **Liquid state machine (LSM):** ESN con neuronas binarias
- ▶ **Otra:** optimización restringida de $\mathbf{W}_{hh}$ para mantenerla ortogonal
- ▶ **Alternativa:** modificar la arquitectura de la RNN con actualizaciones aditivas de los estados ocultos, en lugar de multiplicativas

15.2.7. Gating y long term memory

- ▶ **“Vanilla” RNNs:** plantean grandes dificultades de entrenamiento a causa del problema de los gradientes que desaparecen
- ▶ **Solución:**
 - ▷ Actualizar los estados ocultos de manera aditiva, al estilo de las redes residuales en visión
 - ▷ **Gated recurrent unit (GRU):** propuesta en 2014, la idea clave consiste en aprender mediante un mecanismo de reinicio o actualización de un estado que llamamos **gating unit**
 - ▷ **Long short term memory (LSTM):** propuesta en 1997 (20 años antes de alcanzar gran popularidad), puede verse como una versión sofisticada de una GRU

15.2.7.1. Gated recurrent units (GRU)

- **Entrada:** $\mathbf{X}_t \in \mathbb{R}^{N \times D}$, con batch y vocabulario de tallas N y D
- **Estado oculto:** $\mathbf{H}_t \in \mathbb{R}^{N \times H}$, con H unidades ocultas
- **Reset gate:** $\mathbf{R}_t \in [0, 1]^{N \times H}$ (en $[0, 1]$ por la sigmoide)

$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r) \quad (27)$$

▷ $\mathbf{W}_{xr}$, $\mathbf{W}_{hr}$ y $\mathbf{b}_r \in \mathbb{R}^{1 \times H}$ son los pesos y sesgos de reset

- **Update gate:** $\mathbf{Z}_t \in [0, 1]^{N \times H}$ (en $[0, 1]$ por la sigmoide)

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z) \quad (28)$$

▷ $\mathbf{W}_{xz}$, $\mathbf{W}_{hz}$ y $\mathbf{b}_z$ son los pesos y sesgos de update

► **Candidato:** $\tilde{\mathbf{H}}_t \in [-1, 1]^{N \times H}$ (en $[-1, 1]$ por la tanh)

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h) \quad (29)$$

▷ Combina la nueva entrada $\mathbf{X}_t$ con la memoria retenida $\mathbf{R}_t \odot \mathbf{H}_{t-1}$

▷ Si retiene memoria, $\mathbf{R}_t \approx 1$, actúa como una RNN

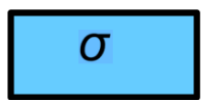
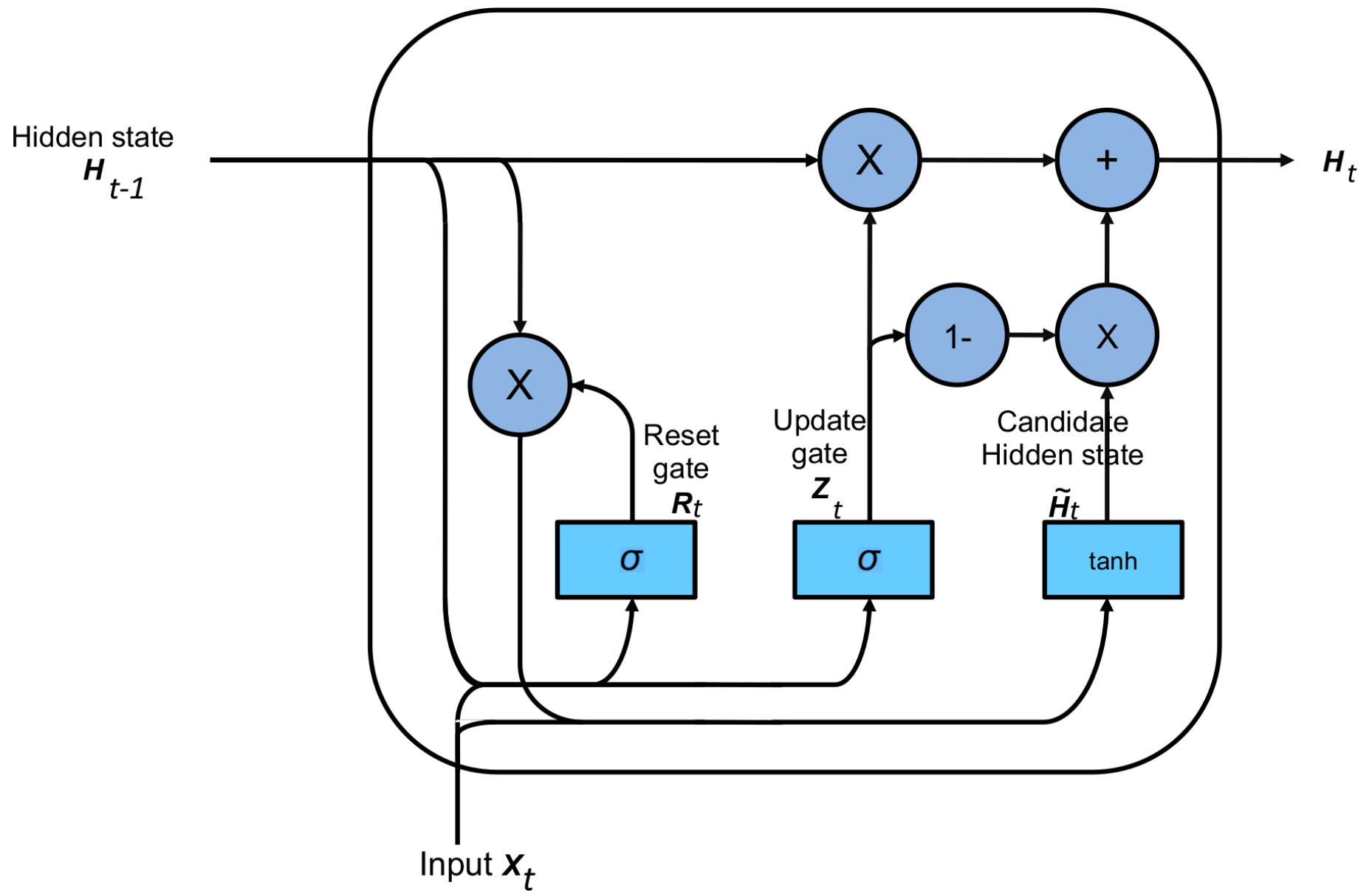
▷ Si no retiene memoria, $\mathbf{R}_t \approx 0$, actúa como un MLP

► **Nuevo:** interpola el estado anterior y el candidato según $\mathbf{Z}_t$

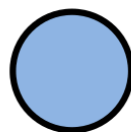
$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t \quad (30)$$

▷ Si $Z_{td} = 1$, se comporta como memoria long-term, $H_{t,d} = H_{t-1,d}$

▷ Si $Z_{td} = 0$, se comporta según el candidato, $H_{t,d} = \tilde{H}_{t,d}$



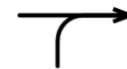
FC layer with
Activation function



Element-wise
Operator



Copy



Concatenate

15.2.7.2. Long short term memory (LSTM)

► **Cell memory/state:** C_t , aumenta el estado oculto H_t

► **Output gate:** O_t (en $[0, 1]$ por la sigmoide)

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \quad (31)$$

▷ W_{xo} , W_{ho} y b_o son los pesos y sesgos de output

► **Input gate:** I_t (en $[0, 1]$ por la sigmoide)

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \quad (32)$$

▷ W_{xi} , W_{hi} y b_i son los pesos y sesgos de input

► **Forget gate:** F_t (en $[0, 1]$ por la sigmoide)

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \quad (33)$$

▷ W_{xf} , W_{hf} y b_f son los pesos y sesgos de forget

- **Celda candidata:** (en $[-1, 1]$ por la tanh)

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c) \quad (34)$$

- ▷ $\mathbf{W}_{xc}$, $\mathbf{W}_{hc}$ y $\mathbf{b}_c$ son los pesos y sesgos de candidata

- **Celda nueva:**

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \quad (35)$$

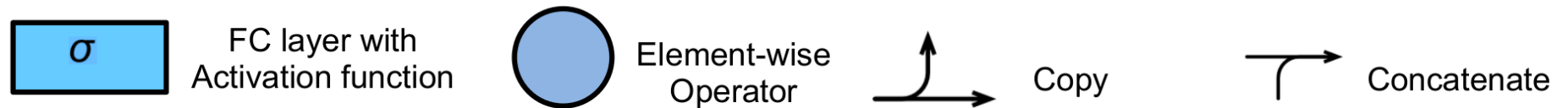
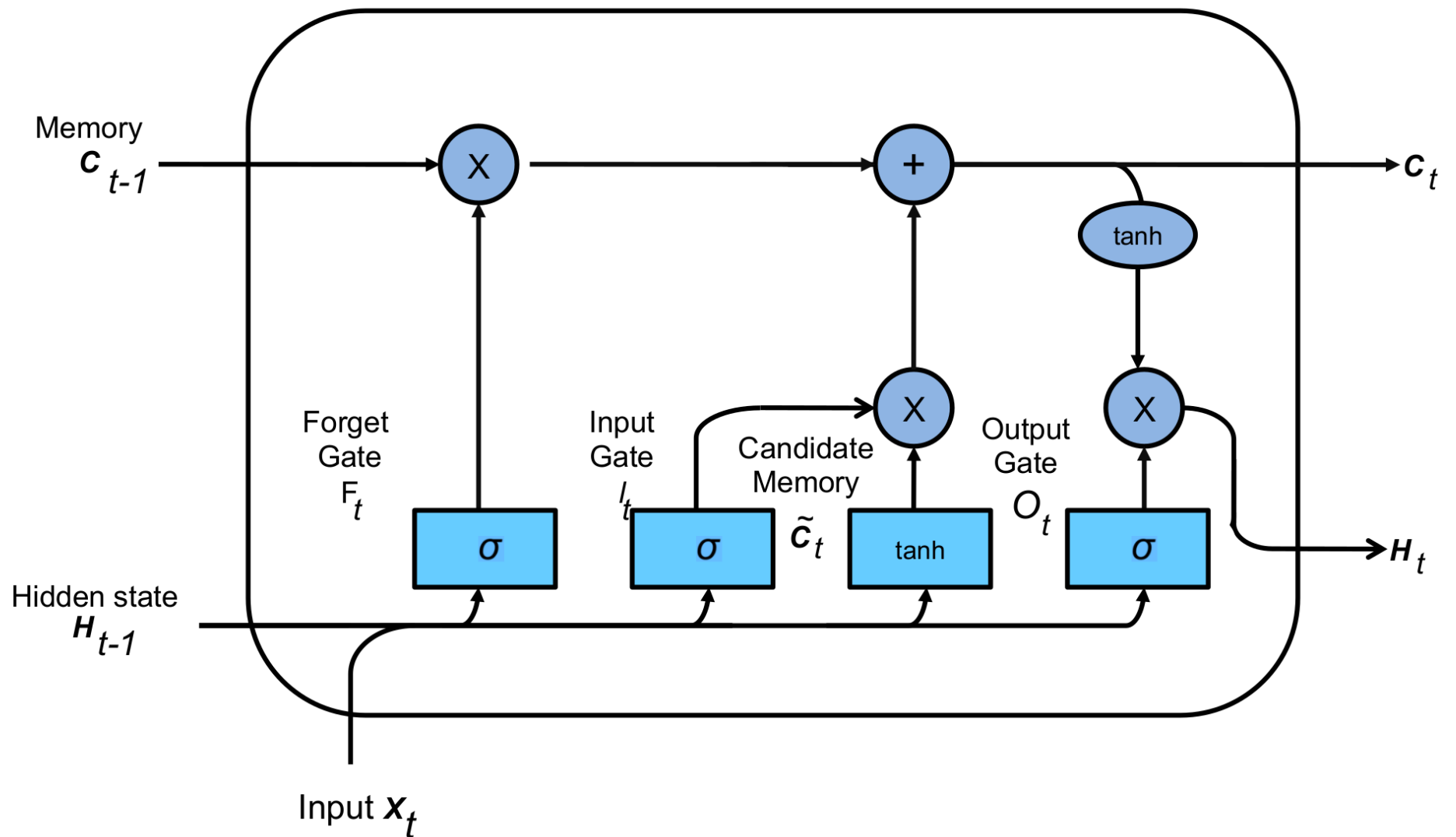
- ▷ Si $\mathbf{I}_t = 0$ y $\mathbf{F}_t = 1$, es memoria long-term, $\mathbf{C}_t = \mathbf{C}_{t-1}$

- **Estado oculto:**

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \quad (36)$$

- ▷ Si $\mathbf{O}_t = 1$, es una transformación inmediata de $\mathbf{C}_t$
- ▷ **Short-term memory:** el modelo recuerda la salida anterior pues $\mathbf{H}_t$ se usa como salida en t y estado oculto (de entrada) en $t + 1$

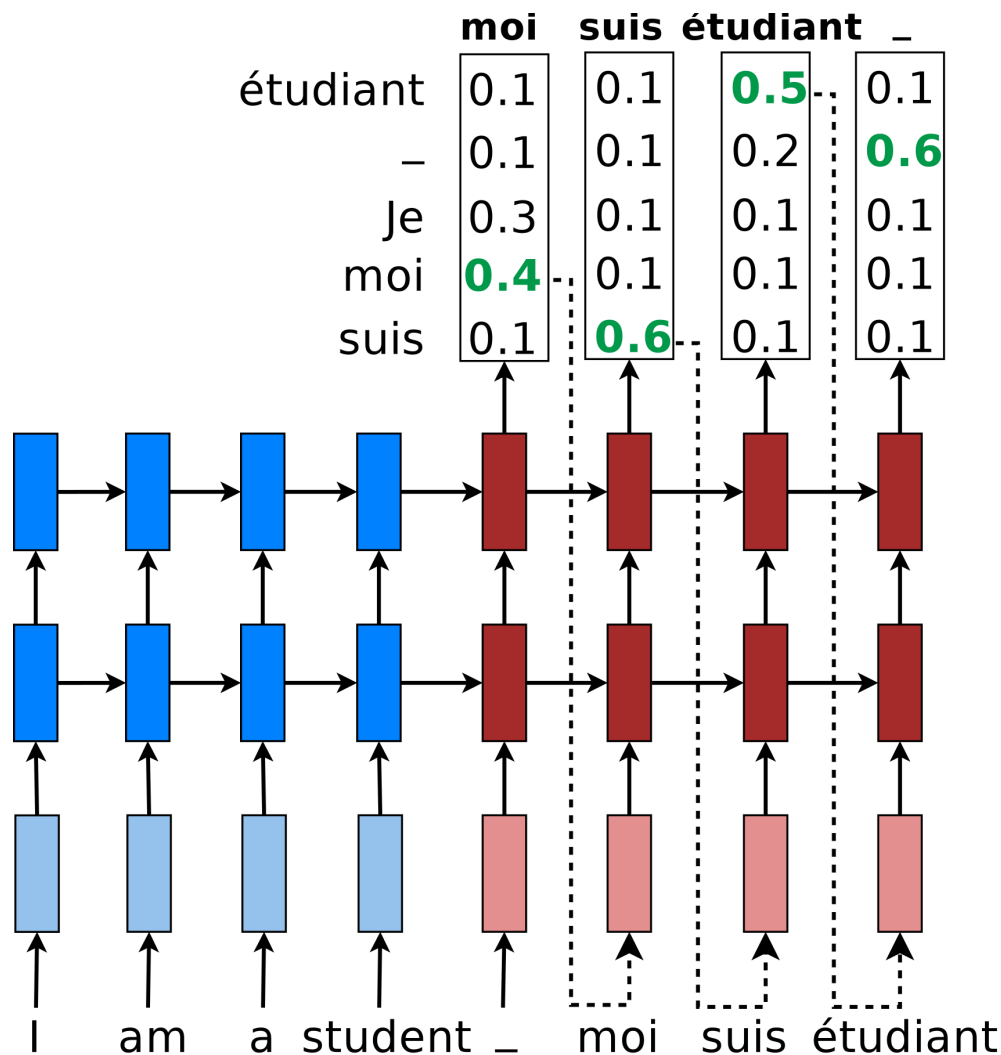
- **Conexiones peephole:** puertas con acceso al estado anterior



15.2.8. Beam search

► *Greedy decoding*: en cada t , calculamos:

$$\hat{y}_t = \arg \max_y p(y_t = y \mid \hat{\mathbf{y}}_{1:t}, \mathbf{x}) \quad (37)$$



► **Secuencia MAP:** (de la que voraz es una aproximación)

$$\mathbf{y}_{1:T}^* = \arg \max_{\mathbf{y}_{1:T}} p(\mathbf{y}_{1:T} \mid \mathbf{x}) \quad (38)$$

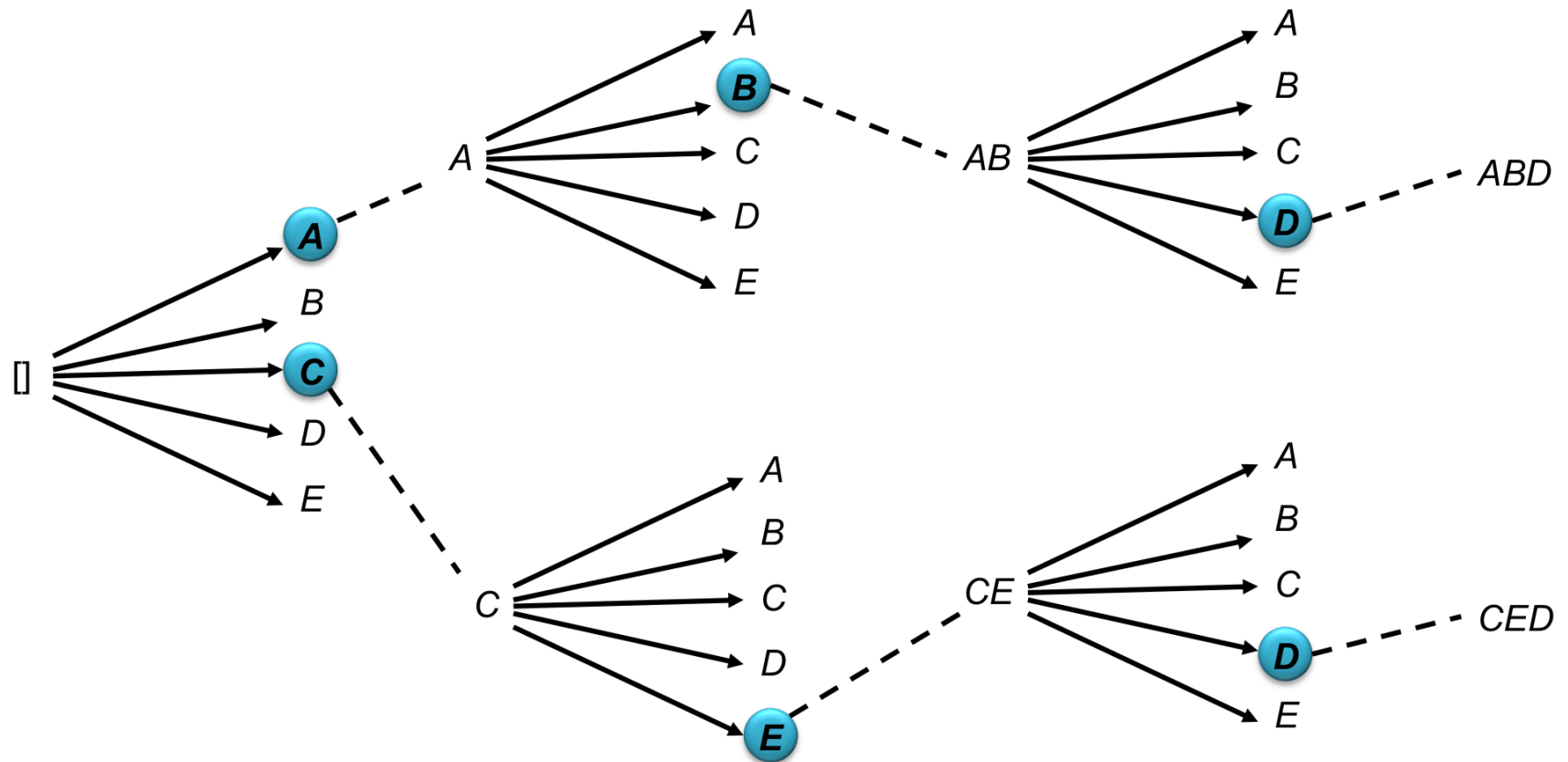
► Ejemplo: greedy 0.048 ABC<eos>; MAP 0.054 ACB<eos>

Time step	1	2	3	4	Time step	1	2	3	4
A	0.5	0.1	0.2	0.0	A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.2	0.2	B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.4	0.2	C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.2	0.6	<eos>	0.1	0.2	0.1	0.6

► Complejidad: $O(V^T)$ con vocabulario de talla V

- **Beam search:** selecciona los K mejores tras generar todos los hijos de los nodos vivos actuales (VK a partir de $t = 2$).

► Ejemplo con $K = 2$:



► Variantes: **stochastic beam search** y **diverse beam search**

15.3. CNNs unidimensionales

- ▶ Las CNNs calculan una función de entorno local para cada entrada usando pesos ligados, y devuelven una salida
- ▶ Aunque las CNNs se suelen usar en 2d para visión, también pueden aplicarse en 1d para secuencias
- ▶ Las CNNs constituyen una alternativa a las RNNs, más fáciles de entrenar pues no deben mantener memoria long-term

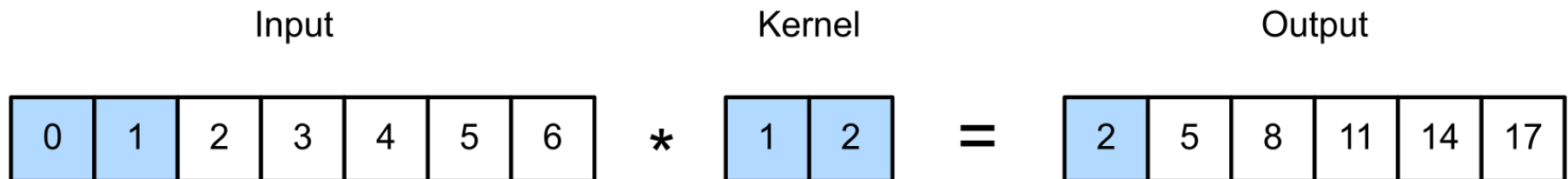
15.3.1. CNNs 1d para clasificación de secuencias

- ▶ Se trata de usar CNNs 1d para aprender funciones de la forma

$$f_{\Theta} : \mathbb{R}^{DT} \rightarrow \mathbb{R}^C \quad (39)$$

donde

- ▷ T es la longitud de la entrada
 - ▷ D es el número de características por entrada
 - ▷ C es la talla del vector de salida (e.g., logits)
- ▶ Ejemplo de convolución 1d aplicada a una secuencia 1d:



► **Formalmente:** si k es la talla del campo receptivo 1d, la salida z puede ser...

▷ $z \in \mathbb{R}^T$ con un filtro w_d por canal de entrada d :

$$z_i = \sum_d \mathbf{x}_{i-k:i+k,d}^t \mathbf{w}_d \quad (40)$$

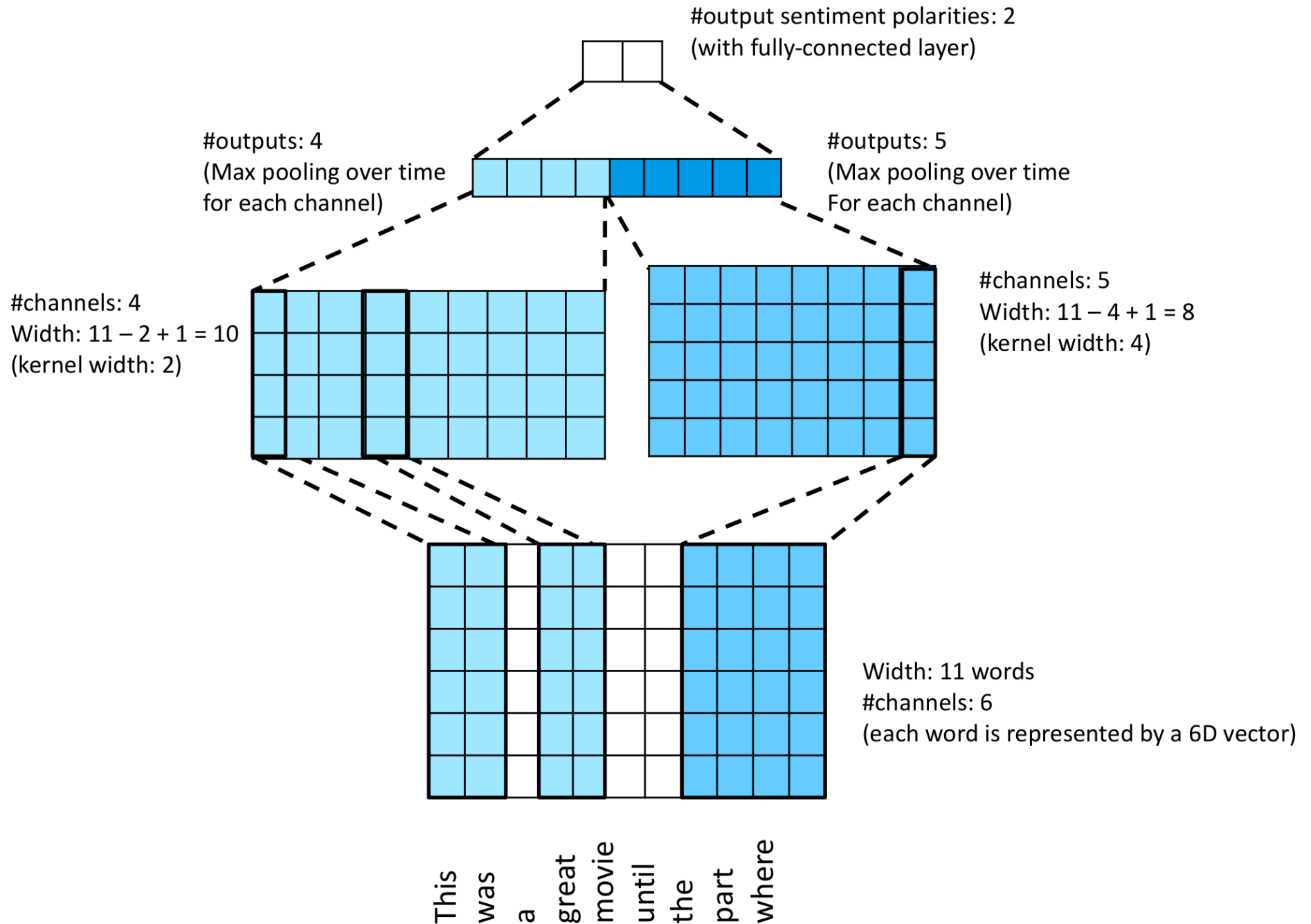
▷ $z \in \mathbb{R}^{TC}$ con múltiples canales de salida:

$$z_{ic} = \sum_d \mathbf{x}_{i-k:i+k,d}^t \mathbf{w}_{dc} \quad (41)$$

▷ $z \in \mathbb{R}^C$ con max-pooling:

$$z_c = \max_i z_{ic} \quad (42)$$

► Modelo TextCNN para clasificación de sentimiento binaria:



15.3.2. CNNs 1d causales para generar secuencias

- *CNN causal o modelo de Markov convolucional*: cada variable de salida solo depende de variables previamente generadas

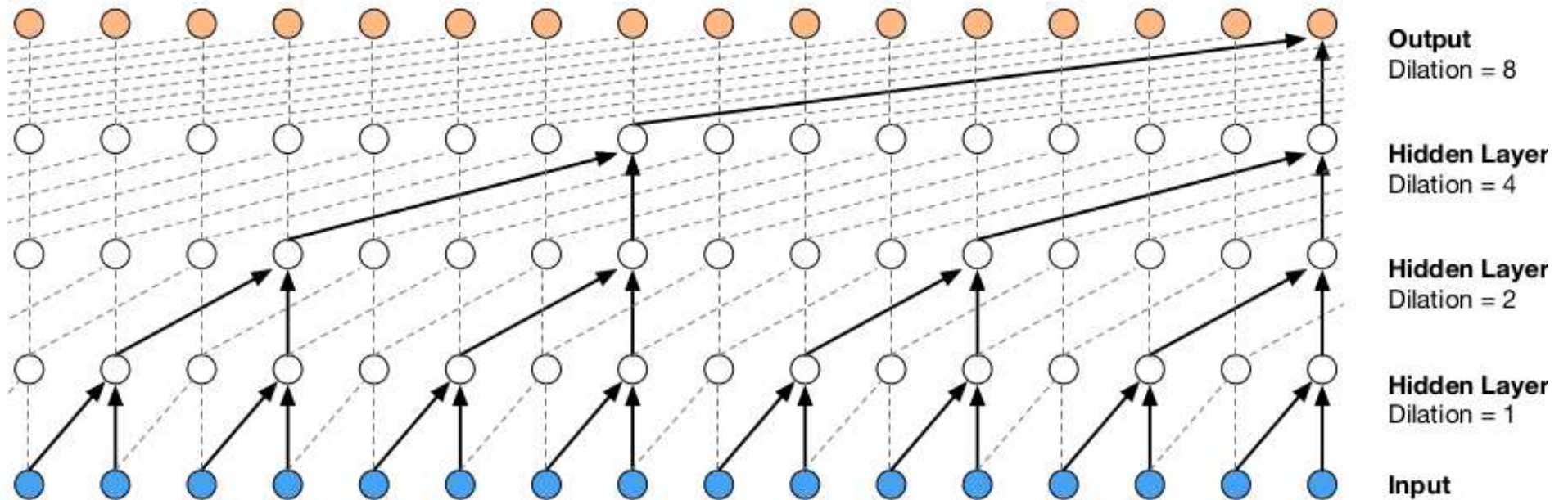
$$p(\mathbf{y}) = \prod_{t=1}^T p(y_t \mid \mathbf{y}_{1:t-1}) \quad (43)$$

$$= \prod_{t=1}^T \mathcal{M}(y_t \mid \mathcal{S}(\varphi(\sum_{\tau=1}^{t-k} \mathbf{w}^t \mathbf{y}_{\tau:\tau+k}))) \quad (44)$$

donde

- ▷ $\mathbf{w}$ es un filtro convolucional de talla k
- ▷ por simplicidad, se asume una única no-linealidad φ y salida categórica

- **Wavenet (TTS):** 10 capas convolucionales causales 1d con ratios de dilatación $1, \dots, 512$; condicionada a características lingüísticas



15.4. Atención

- ▶ En las redes clásicas, las activaciones ocultas son combinaciones lineales de las entradas, seguidas de una no-linealidad

$$z = \varphi(\mathbf{W}v) \quad (45)$$

donde

- ▷ $v \in \mathbb{R}^v$ es un valor de entrada (oculto)
- ▷ $\mathbf{W} \in \mathbb{R}^{v' \times v}$ es un conjunto de pesos aprendido y fijo

▶ *Atención:*

- ▷ Disponemos de m **valores** de entrada $\mathbf{V} \in \mathbb{R}^{m \times v}$
- ▷ El modelo escoge (dinámicamente) un valor de entrada por similitud entre una **query** $q \in \mathbb{R}^q$ y m **keys** $\mathbf{K} \in \mathbb{R}^{m \times k}$

15.4.1. Atención: búsqueda suave en diccionario

► *Atención como búsqueda suave en diccionario*

- ▷ Comparamos la consulta (query) q con cada clave (key) k_i y recuperamos el valor correspondiente v_i
- ▷ Para hacer la búsqueda diferenciable, en lugar de recuperar un único valor, hallamos una combinación convexa de los valores:

$$\text{Attn}(q, (k_{1:m}, v_{1:m})) = \sum_{i=1}^m \alpha_i(q, k_{1:m}) v_i \in \mathbb{R}^v \quad (46)$$

donde los *pesos de atención*, $\{\alpha_i(q, k_{1:m})\}$ satisfacen

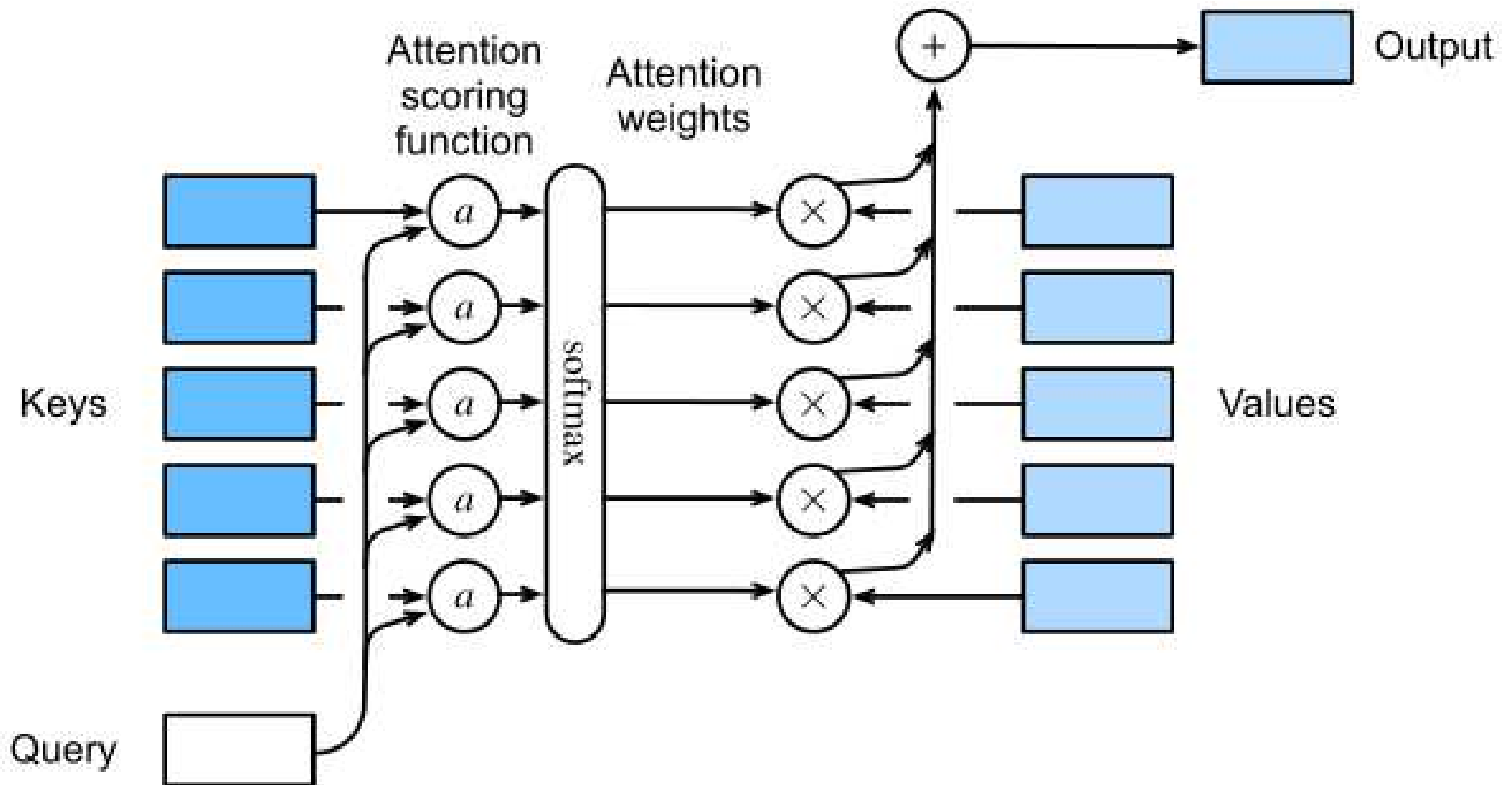
$$0 \leq \alpha_i(q, k_{1:m}) \leq 1 \quad \text{y} \quad \sum_i \alpha_i(q, k_{1:m}) = 1 \quad (47)$$

- ▷ Los pesos de atención pueden hallarse mediante una función **score de atención** $a(\mathbf{q}, \mathbf{k}_i) \in \mathbb{R}$ que calcula la similitud de la consulta $\mathbf{q}$ con cada clave $\mathbf{k}_i$

$$\alpha_i(\mathbf{q}, \mathbf{k}_{1:m}) = \mathcal{S}_i([a(\mathbf{q}, \mathbf{k}_1), \dots, a(\mathbf{q}, \mathbf{k}_m)]) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))} \quad (48)$$

⇒ **Masked attention:** si queremos restringir la atención a un subconjunto del diccionario, podemos fijar el score de atención de las entradas a ignorar (“masked out”) con un valor negativo grande (-10^6)

► *Representación gráfica del mecanismo de atención:*



15.4.2. Regresión kernel: atención no paramétrica

► **Regresión kernel:** la muestra de test x es la query, las de training $x_{1:n}$ son las claves y las etiquetas de training $y_{1:n}$ son los valores

$$f(x) = \sum_{i=1}^n \alpha_i(x, x_{1:n}) y_i \quad (49)$$

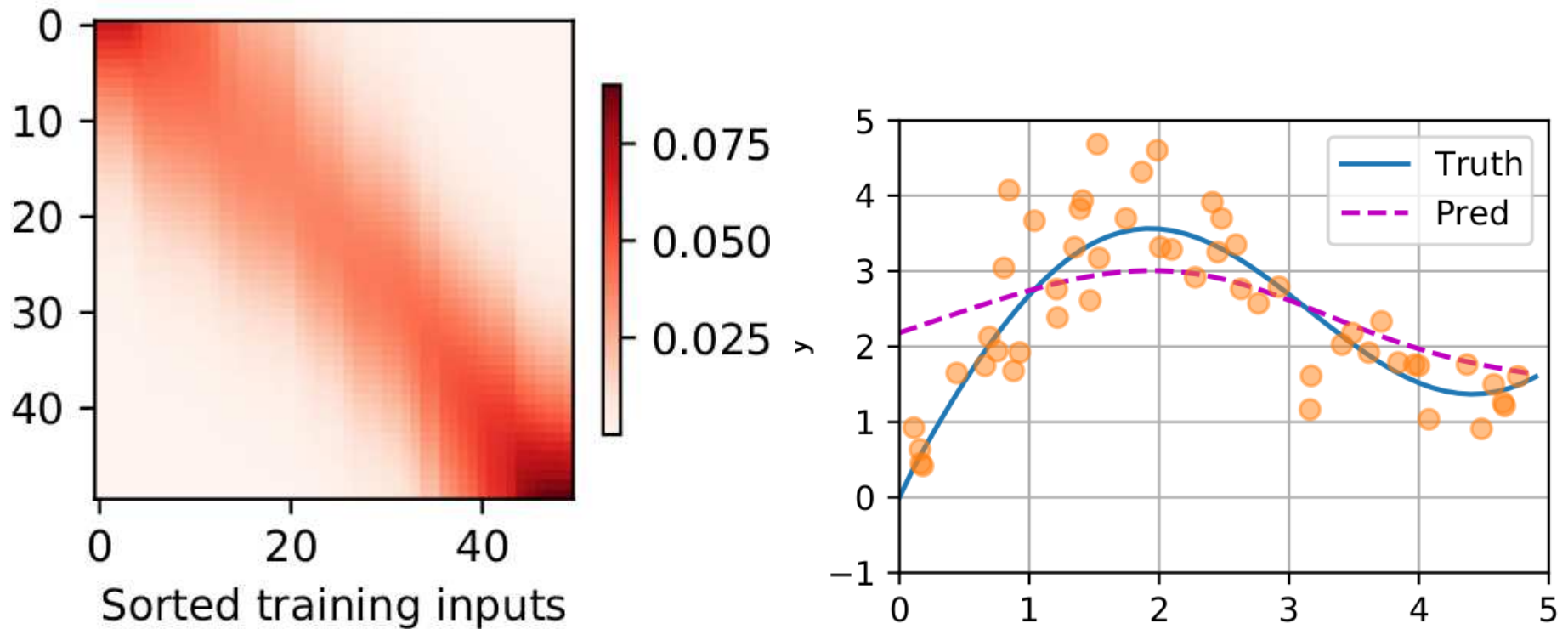
$$= \sum_{i=1}^n \frac{\exp[-\frac{1}{2}((x - x_i)w)^2]}{\sum_{j=1}^n \exp[-\frac{1}{2}((x - x_j)w)^2]} y_i \quad (50)$$

$$= \sum_{i=1}^n \mathcal{S}_i \left[-\frac{1}{2}((x - x_1)w)^2, \dots, -\frac{1}{2}((x - x_n)w)^2 \right] y_i \quad (51)$$

donde $\alpha_i(x, x_{1:n})$ mide la similitud normalizada entre x y x_i , a partir de scores calculados mediante una medida de similitud definida con un kernel de densidad Gaussiana, tomando $w = \frac{1}{\sigma}$,

$$a(x, x_i) = \mathcal{K}_\sigma(x - x_i) \quad \text{con} \quad \mathcal{K}_\sigma(u) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}u^2} \quad (52)$$

- *Ejemplo:* regresión kernel en 1d, con $w = 1$
 - ▷ Izquierda: matriz $A_{ji} = \alpha_i(x, x_{1:n})$ para la entrada de test j
 - ▷ Derecha: curva predicha



- ▷ A mayor w , menor ancho de banda del kernel, σ , por lo que la banda se estrecha y el modelo se sobre-ajusta

15.4.3. Atención paramétrica

- **Atención aditiva:** dada una query $\mathbf{q} \in \mathbb{R}^q$ y una clave $\mathbf{k} \in \mathbb{R}^k$, la función de scoring mediante atención aditiva es:

$$\alpha(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^t \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}) \in \mathbb{R} \quad (53)$$

con parámetros $\mathbf{W}_q \in \mathbb{R}^{h \times q}$, $\mathbf{W}_k \in \mathbb{R}^{h \times k}$ y $\mathbf{w}_v \in \mathbb{R}^h$

- Equivale a un MLP con $\mathbf{q} \in \mathbb{R}^q$ y $\mathbf{k} \in \mathbb{R}^k$ concatenados de entrada y una capa oculta de h unidades sin sesgos

- **Atención producto escalar escalado:** si $\mathbf{q}, \mathbf{k} \in \mathbb{R}^d$ con media nula y varianza identidad, su producto escalar escalado por $1/\sqrt{d}$ también exhibe media nula y varianza unitaria:

$$\alpha(\mathbf{q}, \mathbf{k}) = \mathbf{q}^t \mathbf{k} / \sqrt{d} \in \mathbb{R} \quad (54)$$

- Con minibatches de talla n : $\mathbf{Q} \in \mathbb{R}^{n \times d}$, $\mathbf{K} \in \mathbb{R}^{m \times d}$, $\mathbf{V} \in \mathbb{R}^{m \times v}$

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathcal{S}(\mathbf{Q}\mathbf{K}^t / \sqrt{d}) \mathbf{V} \in \mathbb{R}^{n \times v} \quad (55)$$

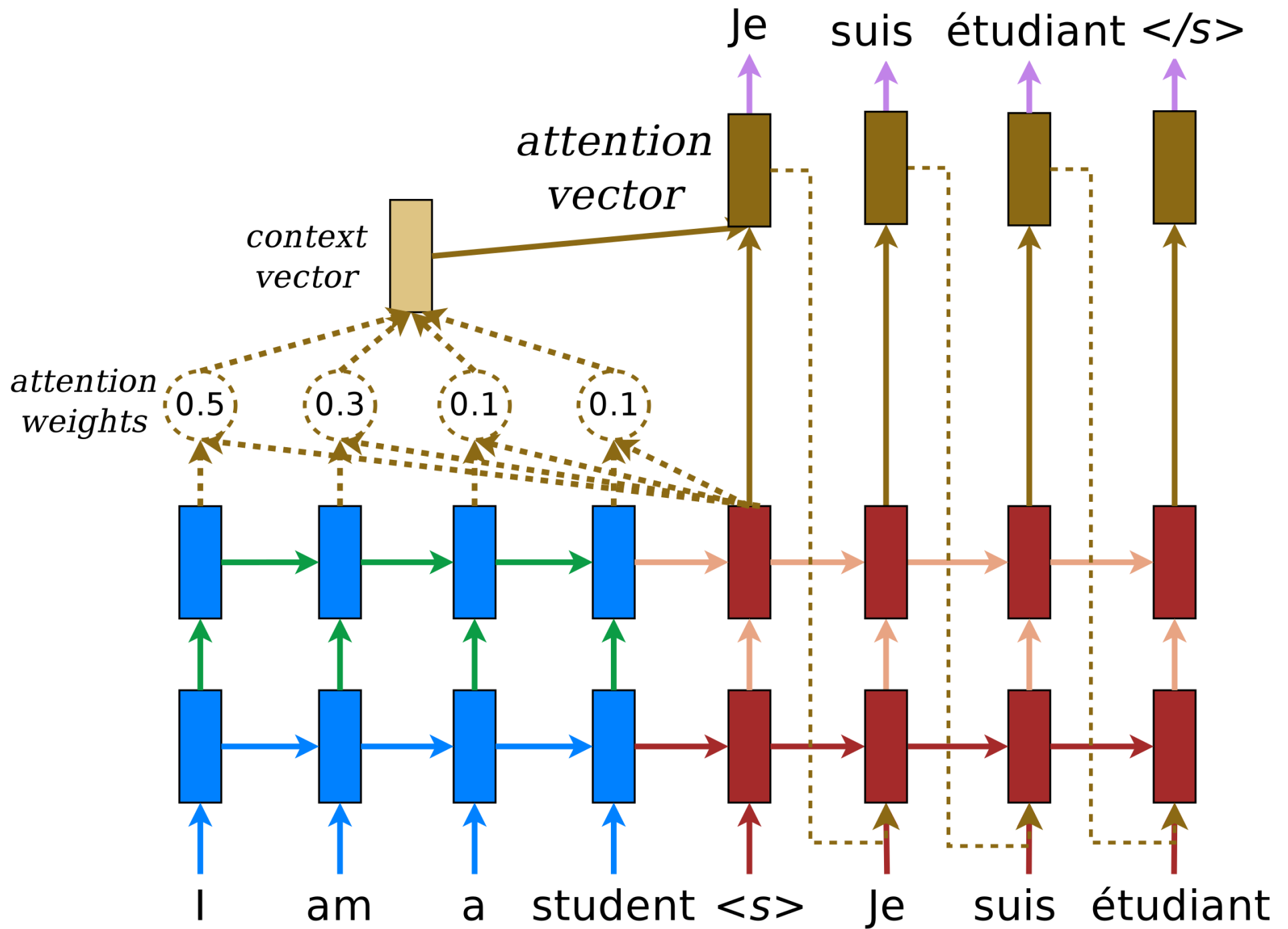
donde la softmax se aplica por filas

15.4.4. Seq2Seq con atención

- ▶ Seq2Seq basado en una RNN con arquitectura encoder-decoder no da buen resultado en MT ya que el decoder no puede acceder directamente a las palabras de entrada, sino que lo hace indirectamente a través de un “cuello de botella” (vector contextual)
- ▶ **Atención en MT:** usa un vector contextual dinámico (no fijo)

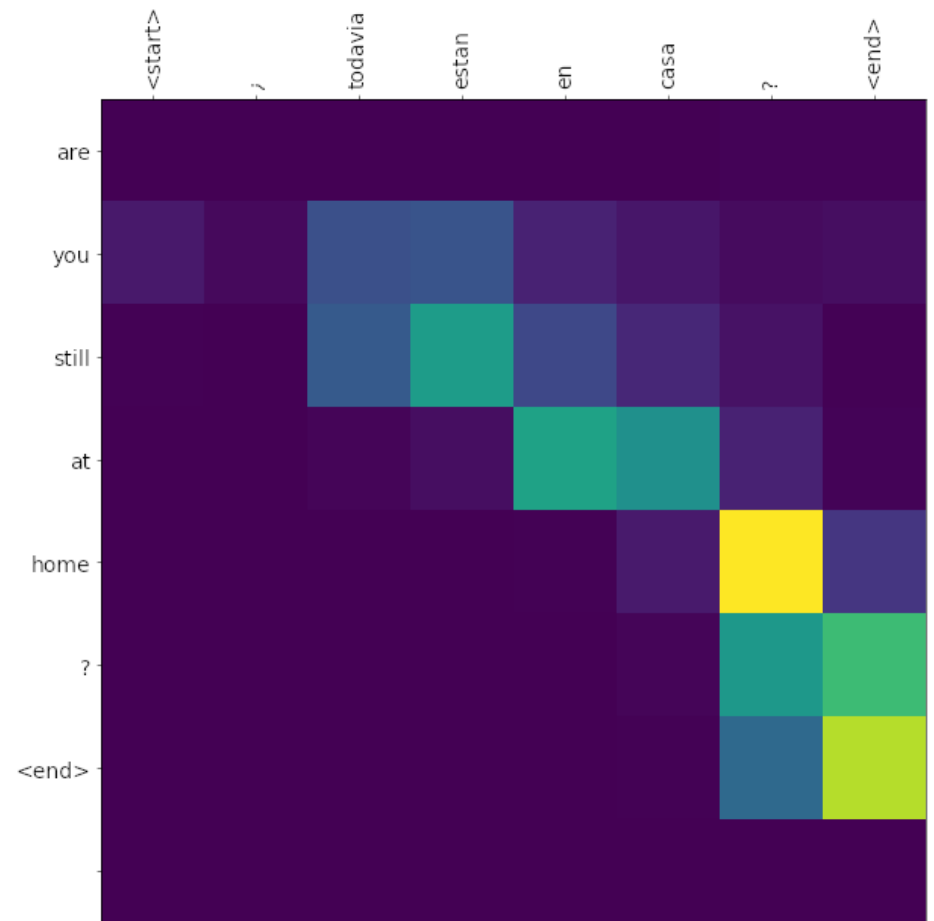
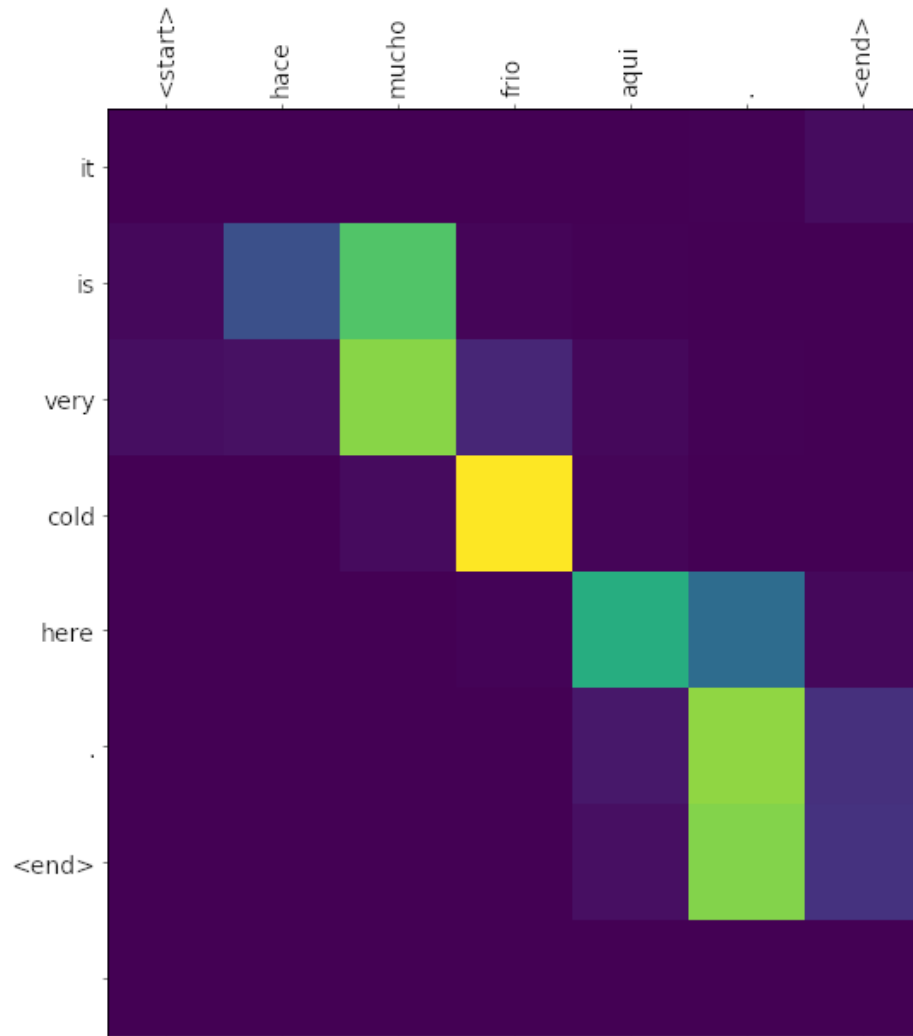
$$\mathbf{c}_t = \sum_{i=1}^T \alpha_i(\mathbf{h}_{t-1}^d, \mathbf{h}_{1:T}^e) \mathbf{h}_i^e \quad (56)$$

- ▷ La query es el estado oculto del decoder en $t - 1$, $\mathbf{h}_{t-1}^d$
- ▷ Las claves son todos los estados ocultos del encoder, $\mathbf{h}_{1:T}^e$
- ▷ Los valores también son los estados ocultos del encoder
- ▷ $\mathbf{c}_t$ se concatena con el vector de entrada del decoder, $\mathbf{y}_{t-1}$, y alimenta el decoder junto con $\mathbf{h}_{t-1}^d$, para crear $\mathbf{h}_t^d$



► *Alineamientos* MT en términos de pesos de atención

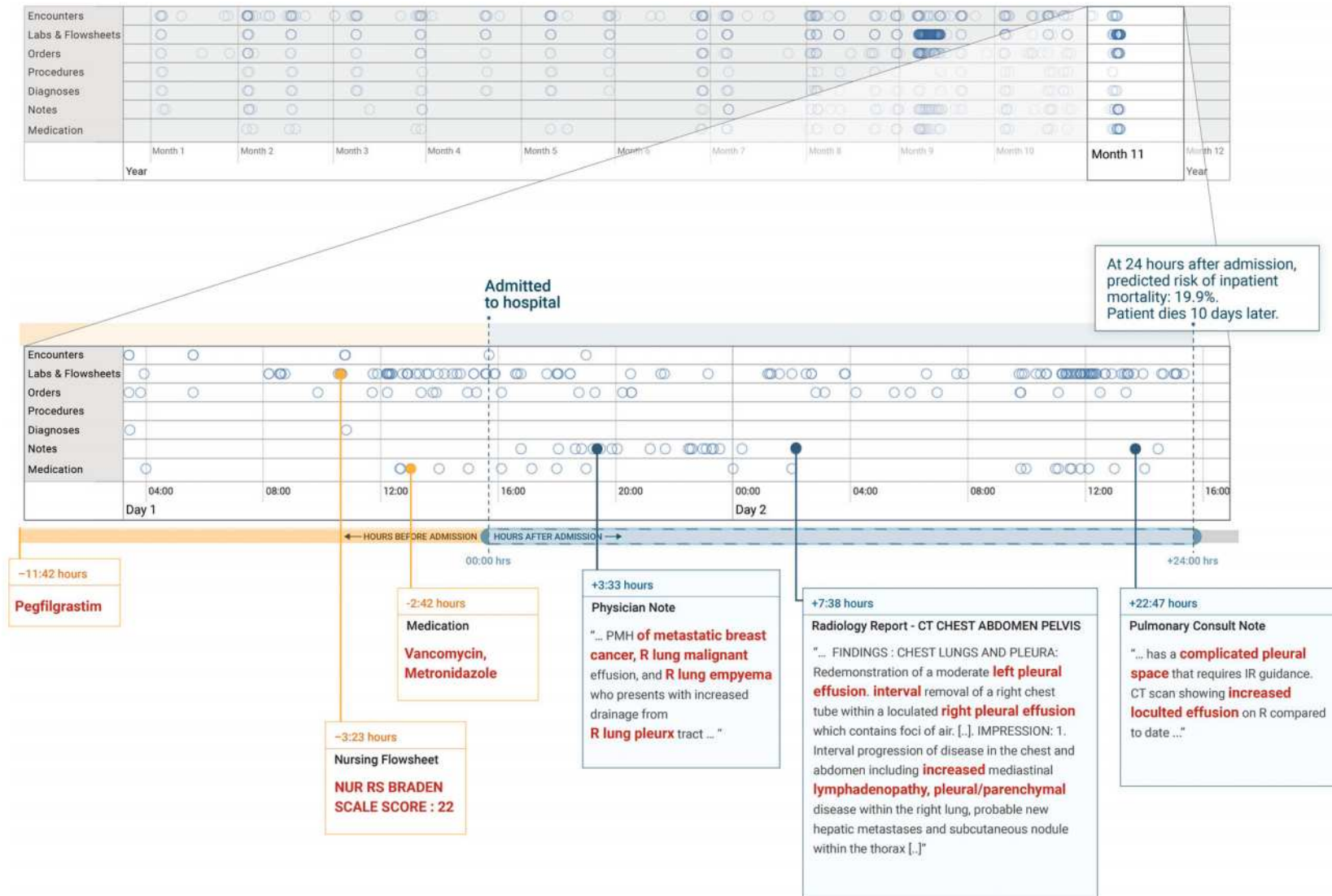
- Izquierda: “hace mucho frío aquí” → “it is very cold here”
- Derecha: “¿todavía están en casa?” → “are you still at home?”



15.4.5. Seq2Vec con atención

► Clasificación de secuencias con atención: registros de salud

Patient Timeline



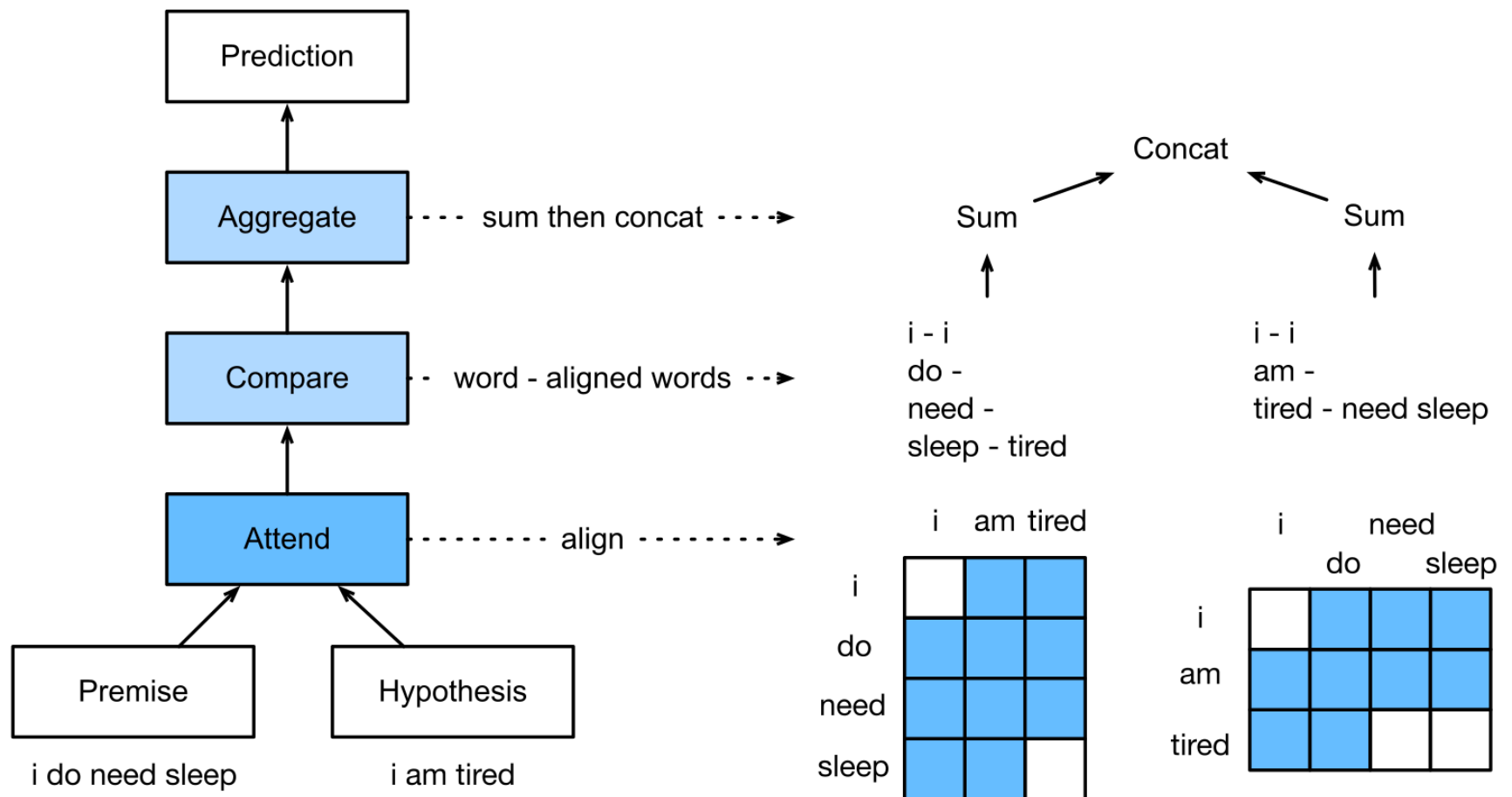
15.4.6. Seq+Seq2Vec con atención

► *Textual entailment o inferencia en lenguaje natural:*

- ▷ Dados dos textos, una premisa y una hipótesis, decimos que
 - ↳ la premisa **implica (entails)** la hipótesis si la hipótesis es verosímil dada la premisa
 - ↳ la premisa **contradice** la hipótesis si la hipótesis no es verosímil dada la premisa
 - ↳ la relación entre ambas es **neutra** si la hipótesis puede darse o no, dada la premisa
- ▷ **Stanford Natural Language Inference (SNLI):** 550K pares de frases etiquetados según las tres categorías anteriores

► *Modelo de 2016 para textual entailment:*

- **Entrada:** premisa $A = (a_1, \dots, a_m)$ e hipótesis $B = (b_1, \dots, b_n)$, donde $a_i, b_j \in \mathbb{R}^E$ son vectores de embedding de las palabras
- **Ejemplo:** clasificación de “I do not sleep” *implica* “I am tired”



- ▷ **Paso 1:** cada palabra en la premisa, $\mathbf{a}_i$, atiende a cada palabra en la hipótesis, $\mathbf{b}_j$, para calcular un peso de atención

$$e_{ij} = f(\mathbf{a}_i)^t f(\mathbf{b}_j) \quad (57)$$

con un MLP $f : \mathbb{R}^E \rightarrow \mathbb{R}^D$ y obtener un promedio ponderado de las palabras que encajan en la hipótesis

$$\beta_i = \sum_{j=1}^n \frac{\exp(e_{ij})}{\sum_{k=1}^m \exp(e_{ik})} \mathbf{b}_j \quad (58)$$

Luego, se compara $\mathbf{a}_i$ con β_i mediante transformación de su concatenación a un espacio oculto con un MLP $g : \mathbb{R}^{2E} \rightarrow \mathbb{R}^H$

$$\mathbf{v}_{A,i} = g([\mathbf{a}_i, \beta_i]) \quad (i = 1, \dots, m) \quad (59)$$

Finalmente, se suman las comparaciones para obtener una similitud global de la premisa a la hipótesis:

$$\mathbf{v}_A = \sum_{i=1}^m \mathbf{v}_{A,i} \quad (60)$$

▷ **Paso 2:** análogamente, la hipótesis se compara con la premisa:

$$\alpha_j = \sum_{i=1}^n \frac{\exp(e_{ij})}{\sum_{k=1}^m \exp(e_{kj})} \mathbf{a}_i \quad (61)$$

$$\mathbf{v}_{B,j} = g([\mathbf{b}_j, \alpha_j]) \quad (j = 1, \dots, n) \quad (62)$$

$$\mathbf{v}_B = \sum_{j=1}^n \mathbf{v}_{B,j} \quad (63)$$

▷ **Paso 3:** se clasifica la salida con otro MLP, $g : \mathbb{R}^{2H} \rightarrow \mathbb{R}^3$

$$\hat{y} = h([\mathbf{v}_A, \mathbf{v}_B]) \quad (64)$$

► **Semantic textual similarity:** tarea relacionada, se trata de predecir el grado de similitud semántica entre dos frases; p.e., en el **benchmark STS** se mide de 0 a 5

15.4.7. Atención blanda y dura

► *Atención dura:*

- ▷ Fuerza mapas de calor de atención dispersos, de manera que cada salida solo puede atender una entrada en lugar de una combinación lineal de todas ellas
- ▷ Conduce a un objetivo de entrenamiento no diferenciable, por lo que requiere métodos como aprendizaje por refuerzo para ajustar el modelo
- ▷ La interpretabilidad de la atención es controvertida

- ▷ Ejemplo de subtitulado de una imagen mediante atención blanda: genera “a woman is throwing a frisbee in a park”



- ▷ Ejemplo de subtitulado de una imagen mediante atención dura: genera “a man and a woman playing frisbee in a field”



15.5. Transformers

- ▶ **Transformer:** modelo seq2seq con atención en encoder y decoder, lo que elimina la necesidad de RNNs
- ▶ Gran aplicabilidad: usados en muchas tareas de generación secuencial (condicional) como
 - ▷ Traducción automática
 - ▷ Análisis de constituencia (constituency parsing)
 - ▷ Generación de música
 - ▷ Generación de secuencias proteínicas
 - ▷ Resumen textual abstractivo
 - ▷ Generación de imágenes

15.5.1. Auto-atención

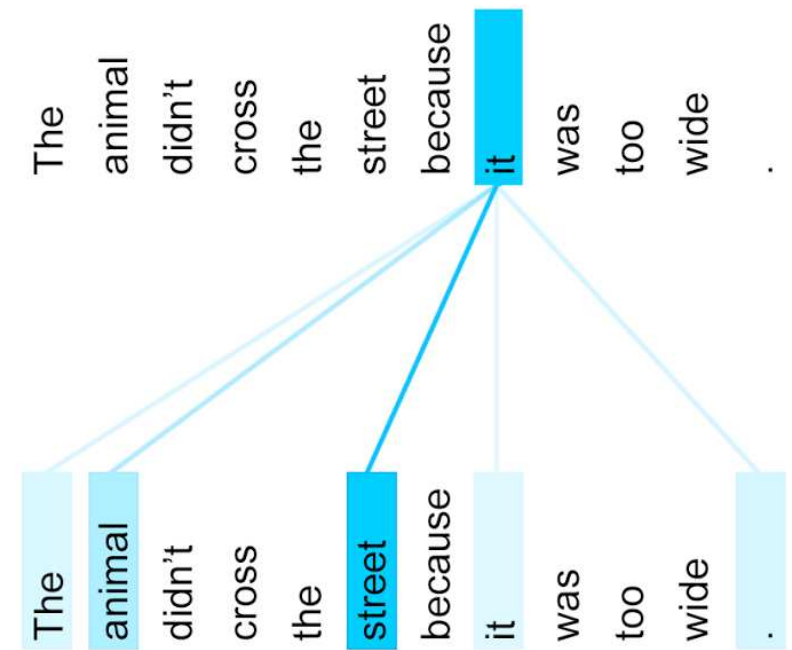
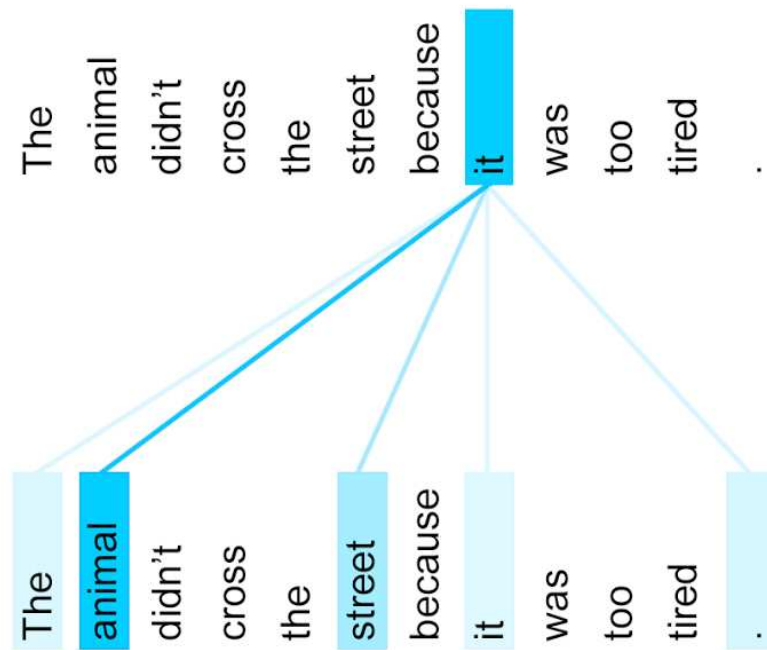
- **Auto-atención en el encoder:** dada una secuencia de tokens de entrada $\mathbf{x}_1, \dots, \mathbf{x}_n$, $\mathbf{x}_i \in \mathbb{R}^d$, auto-atención genera una secuencia de salida de la misma longitud

$$\mathbf{y}_i = \text{Attn}(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n)) \quad (65)$$

donde la consulta es $\mathbf{x}_i$, y las claves y valores son todas las entradas (válidas) $\mathbf{x}_1, \dots, \mathbf{x}_n$

- **Auto-atención en el decoder:** tomamos $\mathbf{x}_i = \mathbf{y}_{i-1}$ y $n = i - 1$ para disponer de todas las salidas previamente generadas
- **Entrenamiento paralelizable:** dado que todas las salidas son conocidas en entrenamiento, el objetivo puede evaluarse en paralelo, cosa que no puede hacerse con RNNs

► Ejemplo de representación de contexto mejorada con auto-atención:



- ▷ Se quiere traducir dos frases distintas al francés
- ▷ Resolución de correferencia: para una traducción correcta, se requiere saber si “it” se refiere a “animal” o “street”
- ▷ Izquierda: auto-atención resuelve que se refiere a “animal”
- ▷ Derecha: auto-atención resuelve que se refiere a “street”

15.5.2. Atención multi-cabezal

- **Multi-headed attention (MHA):** introduce múltiples matrices (kernels) de atención para capturar diferentes nociones de similitud
- Dada una consulta $\mathbf{q} \in \mathbb{R}^{d_q}$, claves $\mathbf{k}_j \in \mathbb{R}^{d_k}$ y valores $\mathbf{v}_j \in \mathbb{R}^{d_v}$, el i -ésimo cabezal de atención es

$$\mathbf{h}_i = \text{Attn}(\mathbf{W}_i^{(q)} \mathbf{q}, \{\mathbf{W}_i^{(k)} \mathbf{k}_j, \mathbf{W}_i^{(v)} \mathbf{v}_j\}) \in \mathbb{R}^{p_v} \quad (66)$$

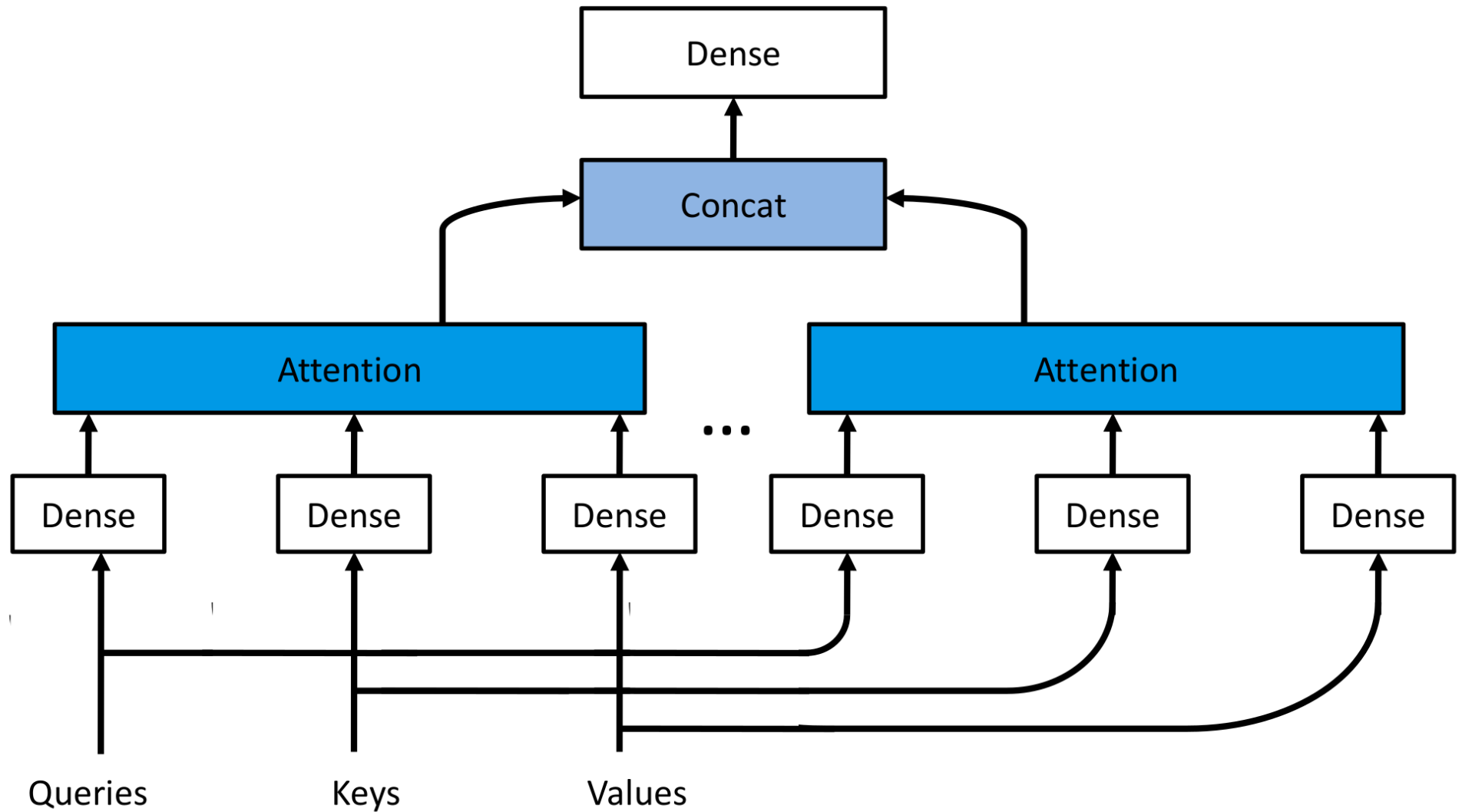
con parámetros $\mathbf{W}_i^{(q)} \in \mathbb{R}^{p_q \times d_q}$, $\mathbf{W}_i^{(k)} \in \mathbb{R}^{p_k \times d_k}$ y $\mathbf{W}_i^{(v)} \in \mathbb{R}^{p_v \times d_v}$

- Los h cabezales se apilan y proyectan en $\mathbb{R}^{p_o}$:

$$\mathbf{h} = \text{MHA}(\mathbf{q}, \{\mathbf{k}_j, \mathbf{v}_j\}) = \mathbf{W}_o \begin{pmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{pmatrix} \in \mathbb{R}^{p_o} \quad (67)$$

con el $\mathbf{h}_i$ dado arriba y $\mathbf{W}_o \in \mathbb{R}^{p_o \times hp_v}$

► Representación gráfica de atención multi-cabezal:



15.5.3. Codificación posicional

► *Positional embedding:*

- ▷ Dificultad: auto-atención es invariante a permutaciones, esto es, ignora el orden de ocurrencia de las palabras
- ▷ Solución: combinar el embedding de palabra con otro posicional

- ▶ Representación de la posición mediante un entero:
 - ▷ Las redes no pueden manejar enteros de forma nativa, pero pueden codificarse en binario
 - ▷ Por ejemplo, para secuencias de longitud $n = 3$, obtenemos la secuencia de vectores binarios de $d = 3$ dimensiones para cada posición: 000, 001, 010, 011, 100, 101, 110, 111
 - ▷ El bit más a la derecha es el que cambia más rápido (con mayor frecuencia), mientras que el de la izquierda es el más lento
 - ▷ Representación: matriz de posición $\mathbf{P} \in \mathbb{R}^{n \times d}$
 - ▷ Puede verse como una representación basada en funciones base (correspondiente a potencias de 2) con coeficientes 0 o 1

- ▶ Transformer emplea una representación basada en funciones base sinusoides con coeficientes reales (C = longitud máxima):

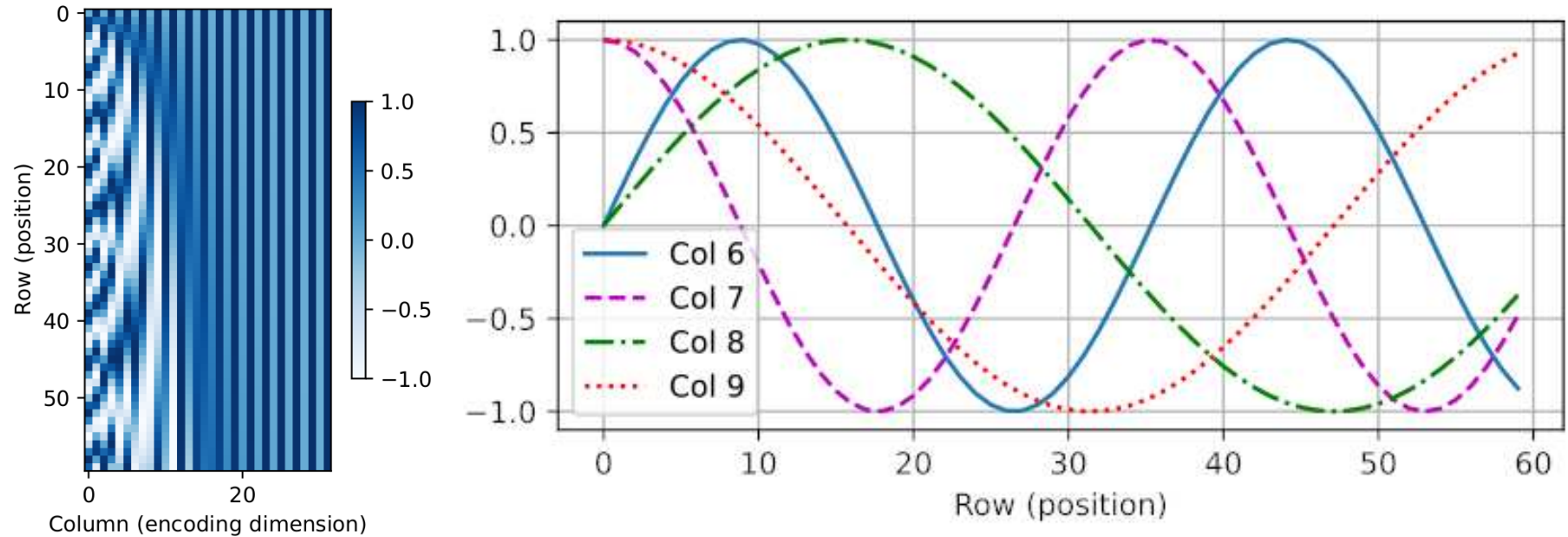
$$p_{i,2j} = \sin \left(\frac{i}{C^{2j/d}} \right) \quad (68)$$

$$p_{i,2j+1} = \cos \left(\frac{i}{C^{2j/d}} \right) \quad (69)$$

- ▶ *Ejemplo:* si $d = 4$, la i -ésima fila es

$$\mathbf{p}_i = \left[\sin \left(\frac{i}{C^{0/4}} \right), \cos \left(\frac{i}{C^{0/4}} \right), \sin \left(\frac{i}{C^{2/4}} \right), \cos \left(\frac{i}{C^{2/4}} \right) \right] \quad (70)$$

► Ejemplo:



- Izquierda: matriz de posición para $n = 60$ y $d = 32$
- Derecha: funciones base para las columnas 6 a 9

► Ventajas de la representación sinusoide:

- ▷ Calculable para entradas de longitud arbitraria hasta $T \leq C$
- ▷ La representación de una posición es linealmente predecible a partir de otra, si se conoce su distancia relativa:

$$\mathbf{p}_{t+\phi} = f(\mathbf{p}_t) \quad (71)$$

siendo f una transformación lineal:

$$\begin{pmatrix} \sin(\omega_k(t + \phi)) \\ \cos(\omega_k(t + \phi)) \end{pmatrix} = \begin{pmatrix} \sin(\omega_k t) \cos(\omega_k \phi) + \cos(\omega_k t) \sin(\omega_k \phi) \\ \cos(\omega_k t) \cos(\omega_k \phi) - \sin(\omega_k t) \sin(\omega_k \phi) \end{pmatrix} \quad (72)$$

$$= \begin{pmatrix} \cos(\omega_k \phi) & \sin(\omega_k \phi) \\ -\sin(\omega_k \phi) & \cos(\omega_k \phi) \end{pmatrix} \begin{pmatrix} \sin(\omega_k t) \\ \cos(\omega_k t) \end{pmatrix} \quad (73)$$

Si ϕ es pequeño, $\mathbf{p}_{t+\phi} \approx \mathbf{p}_t$

- Combinación del embedding posicional $\mathbf{P}$ con el de palabras, $\mathbf{X}$:

$$\text{POS}(\text{Embed}(\mathbf{X})) = \mathbf{X} + \mathbf{P} \quad (74)$$

- Podría hacerse por concatenación, pero se prefiere la suma

15.5.4. Todo junto

► **Encoder:** N copias de bloques de codificación

```
1 def EncoderBlock(X) :  
2     Z = LayerNorm(MultiHeadAttn(Q=X, K=X, V=X) + X)  
3     E = LayerNorm(FeedForward(Z) + Z)  
4     return E
```

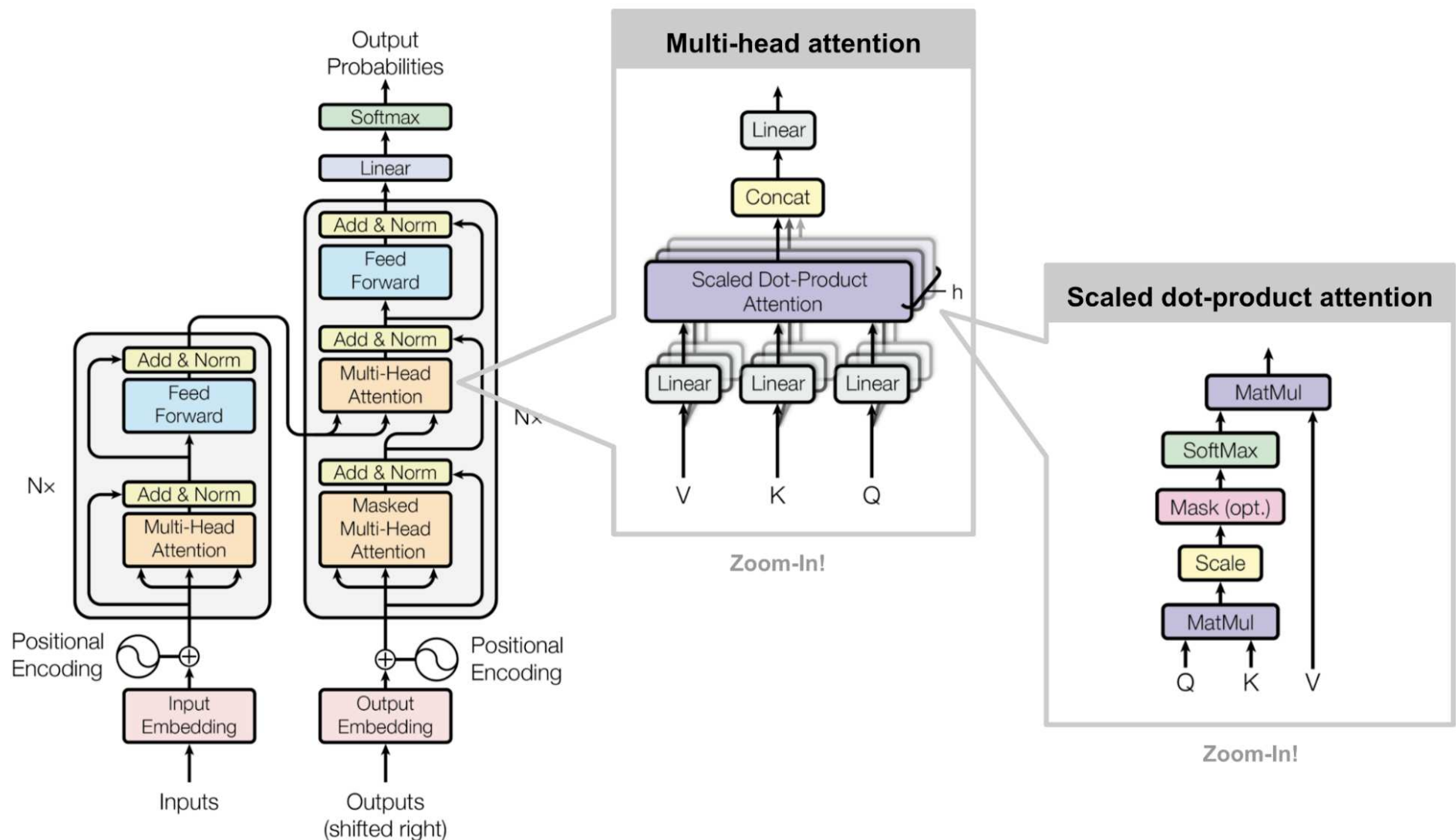
```
1 def Encoder(X, N) :  
2     E = POS(Embed(X))  
3     for n in range(N) :  
4         E = EncoderBlock(E)  
5     return E
```


► **Decoder:** N copias de bloques de decodificación

```
1 def DecoderBlock(Y, E):
2     Z = LayerNorm(MultiHeadAttn(Q=Y, K=Y, V=Y) + Y)
3     ZZ = LayerNorm(MultiHeadAttn(Q=Z, K=E, V=E) + Z)
4     D = LayerNorm(FeedForward(ZZ) + ZZ)
5     return D
```

```
1 def Decoder(Y, E, N):
2     D = POS(Embed(Y))
3     for n in range(N):
4         D = DecoderBlock(D, E)
5     return D
```

► Arquitectura del modelo Transformer “vanilla” (básico):

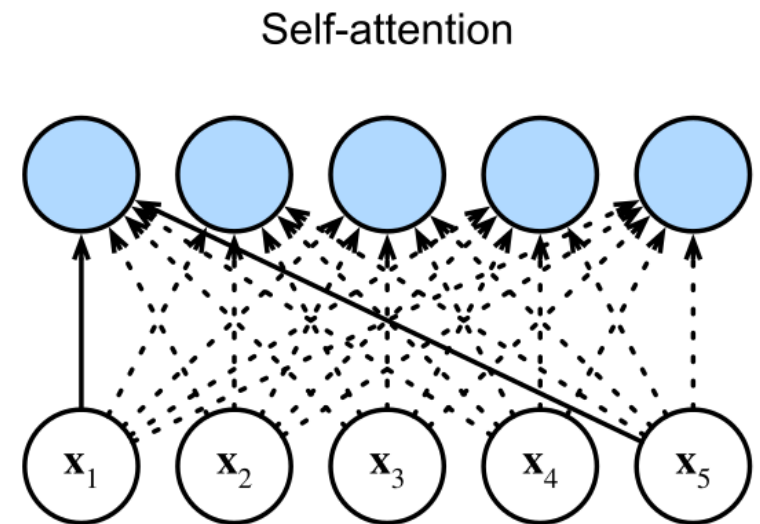
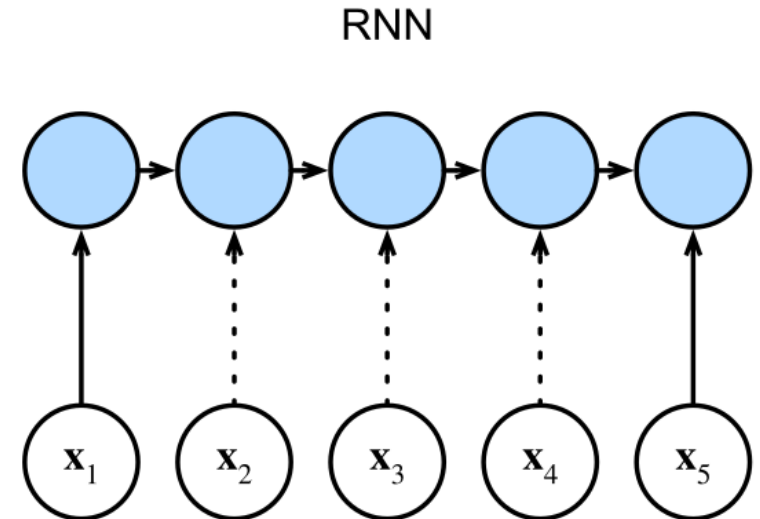
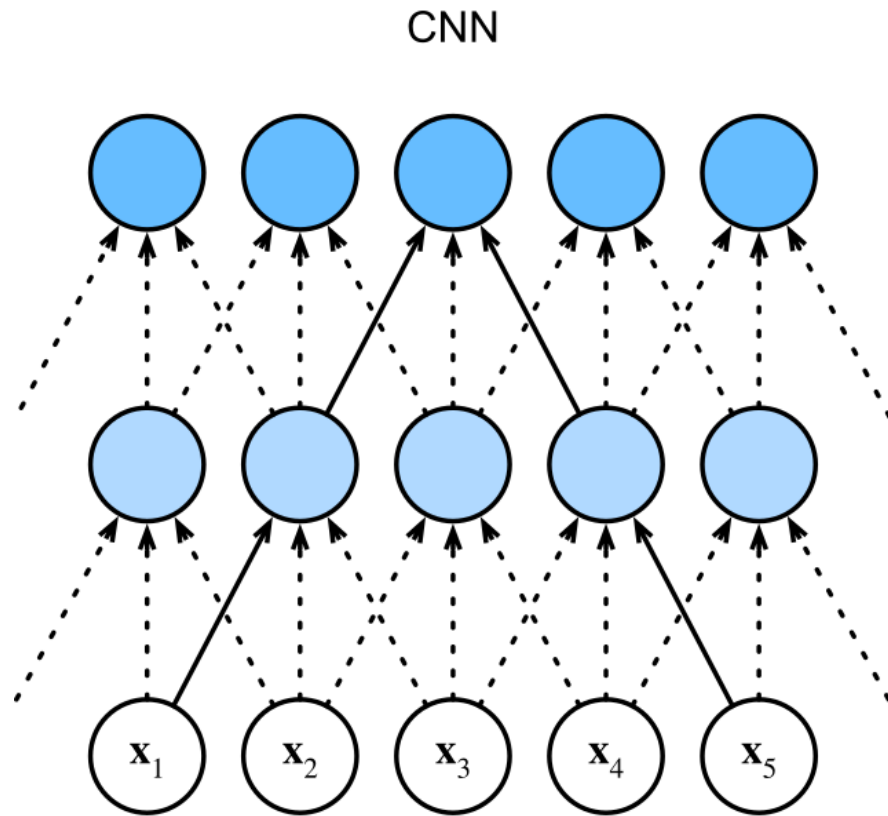


► Entrenamiento: el decoder dispone de todas las entradas Y

► Inferencia: decodificación secuencial, con masked atención

15.5.5. Comparación de transformers, CNNs y RNNs

- Arquitecturas para transformar una secuencia $x_{1:n}$ en otra $y_{1:n}$:



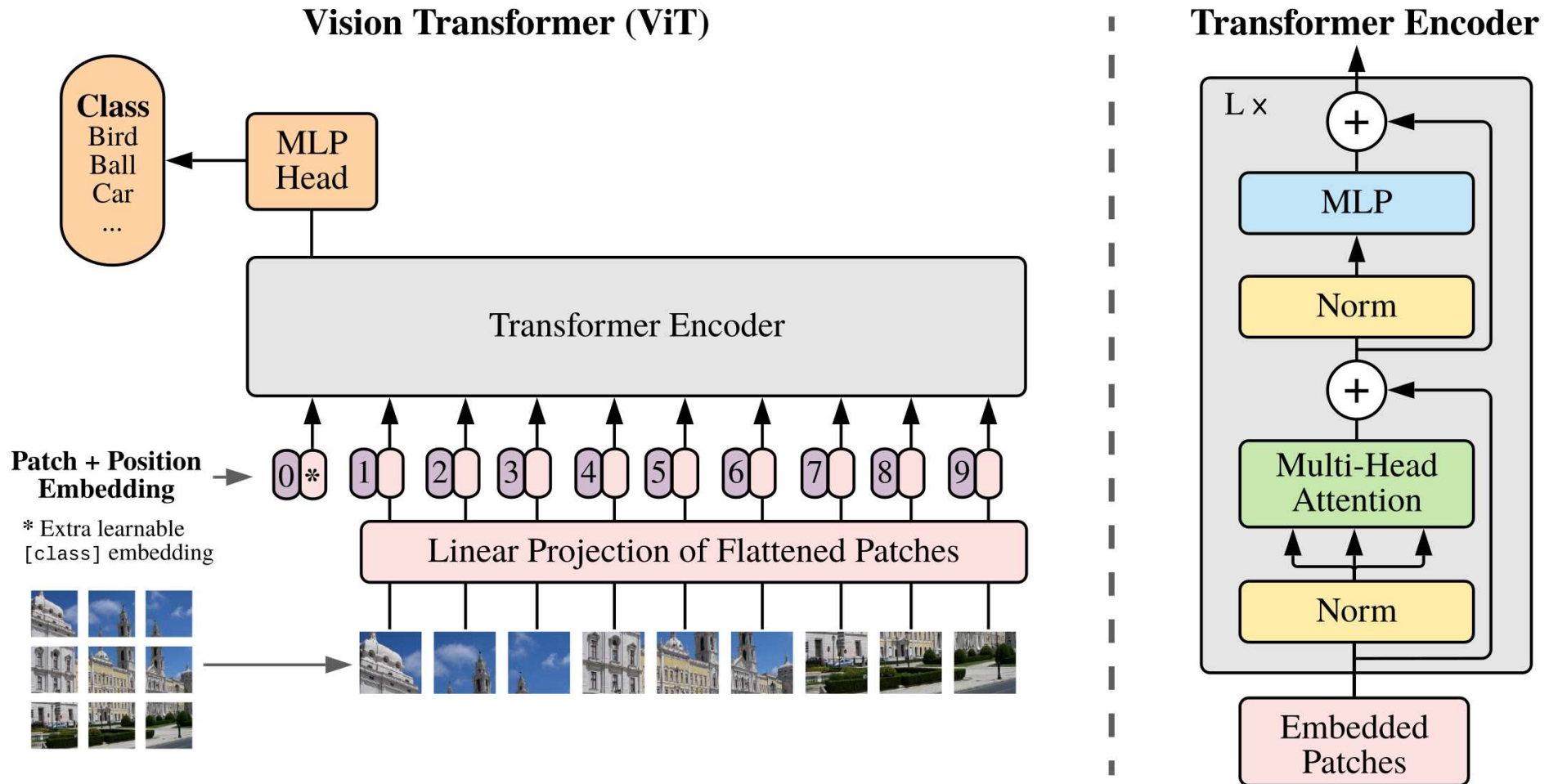
- Comparación: n es la longitud de la secuencia, d la dimensión de las características de entrada y k la talla del kernel convolucional

Capa	Complejidad	Operaciones secuenciales	Max. longitud de camino
Auto-atención	$O(n^2d)$	$O(1)$	$O(1)$
Recurrente	$O(nd^2)$	$O(n)$	$O(n)$
Convolucional	$O(knd^2)$	$O(1)$	$O(\log_k n)$

- 1d CNN: $O(knd^2)$ para calcular la salida en paralelo, aunque se necesita una pila de n/k capas, o $\log_k n$ con dilataciones, para comunicar todos los pares (figura: x_1 y x_5 conectan en la capa 2)
- RNN: $O(nd^2)$ por producto matriz-vector en cada paso secuencial
- Auto-atención: $O(1)$ máxima longitud de camino ya que cada salida se conecta directamente a cada entrada; el coste $O(n^2d)$ es asumible para secuencias cortas, pero no para largas

15.5.6. Transformers para imágenes

- **Vision Transformer (ViT):** para clasificación de imágenes interpretadas como secuencias de trozos de imagen (image patches)



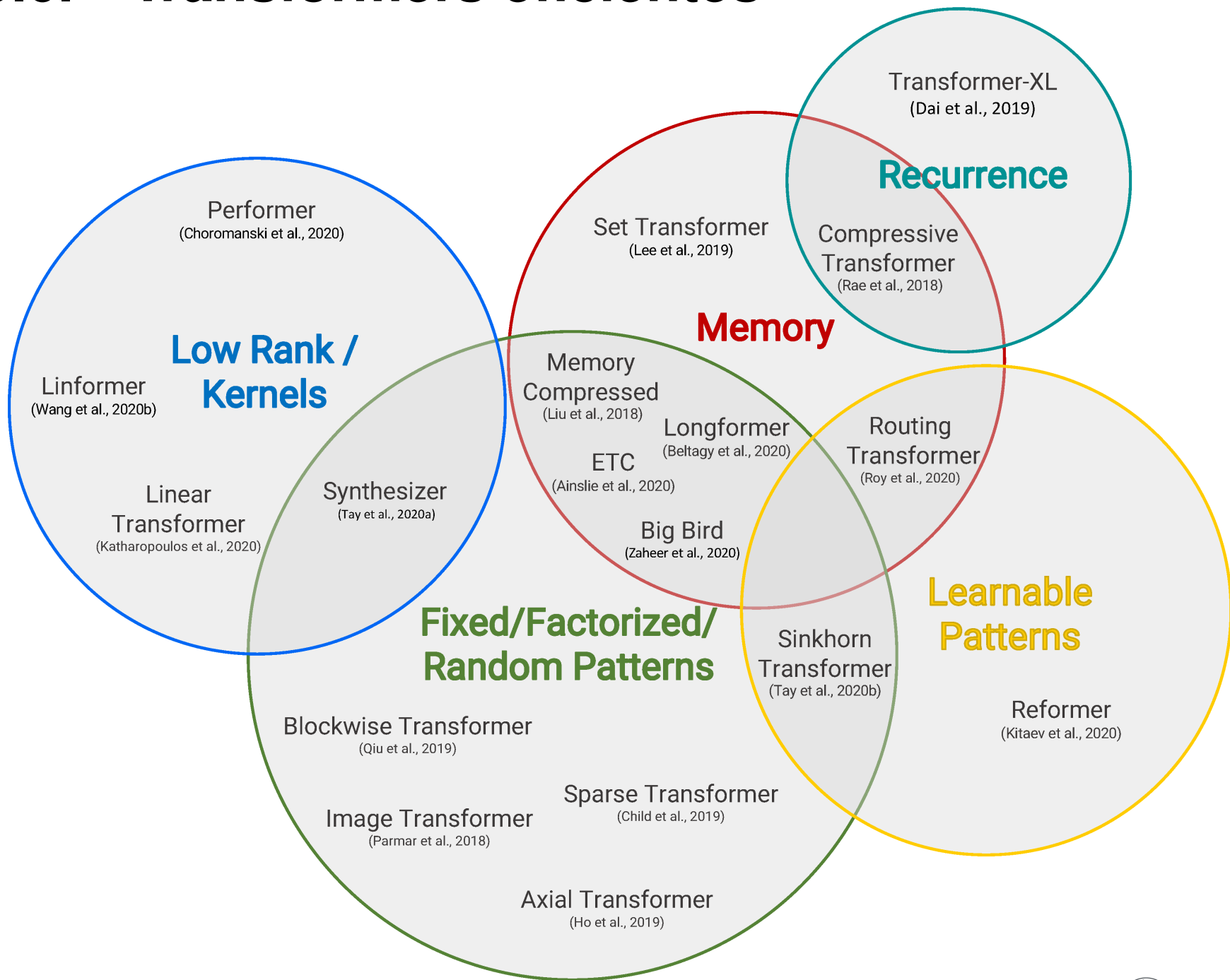
- Muy popular actualmente:

https://github.com/google-research/vision_transformer

15.5.7. Otras variantes transformer

- ▶ *Scaling Giant Models with Conditional Computation and Automatic Sharding (GShard):*
 - ▷ Como su nombre indica, permite escalar modelos gigantes mediante computación condicional y fragmentado automático
 - ▷ Reemplaza las capas densas hacia adelante por un módulo de regresión basado en una mixtura de expertos
- ▶ *Convolution-augmented Transformer for Speech Recognition (Conformer):* añade capas convolucionales dentro de la arquitectura transformer

15.6. Transformers eficientes



15.6.1. Patrones de atención localizados y fijos

- ▶ El mecanismo de atención se restringe mediante una ventana localizada no aprendible y fija, esto es, cada token solo atiende a un conjunto pre-seleccionado de tokens
- ▶ *Ejemplo:* cada secuencia se divide en K bloques de longitud $\frac{N}{K}$ y la atención se lleva a cabo dentro de cada bloque, por lo que la complejidad espacial y temporal se reduce de $O(N^2)$ a $O(\frac{N^2}{K})$
- ▶ Los patrones de atención pueden no ser solo bloques, sino que también son posibles otras aproximaciones como ventanas a saltos o dilatadas, o patrones híbridos combinados

15.6.2. Patrones de atención dispersos y aprendibles

- ▶ Como en los patrones localizados y fijos, el mecanismo de atención se restringe a pares de tokens dentro de una única partición de algún particionamiento de todos los tokens, pero dichos particionamientos no son fijos, sino que se entrenan
- ▶ **Hashing:** los tokens se hashean y cada cubo de hashing corresponde a una partición distinta
- ▶ **Reformer:** aplica **Locality Sensitive Hashing (LSH)** con coste temporal $O(NM^2 \log(M))$ para embeddings de dimensión M
- ▶ **Clustering:** aplica K -means para reducir el coste a $O(\frac{N^2}{K})$ si los clústers son del mismo tamaño; usualmente, $K = \sqrt{N}$

15.6.3. Métodos recurrentes y memorísticos

- ▶ **Memorísticos:** introducen un módulo de memoria adicional, usualmente un algoritmo de **memoria global**, que puede acceder a varios tokens simultáneamente
- ▶ **Recurrence methods:** conectan bloques locales diferentes mediante una recurrencia
- ▶ **Transformer-XL:** clase de métodos recurrentes destacada

15.6.4. Métodos basados en kernels y de bajo rango

- La matriz de atención $\mathbf{A}$ se aproxima con una de bajo rango:

$$A_{ij} = \phi(\mathbf{q}_i)^t \phi(\mathbf{k}_j) \quad (75)$$

donde $\phi(\mathbf{x}) \in \mathbb{R}^M$ es un vector finito-dimensional con $M < D$

- Coste $\mathbf{AV}$: $O(N)$, aunque atención softmax no es de bajo rango
- **Linformer**: transforma claves y valores mediante proyecciones Gaussianas aleatorias y luego aplica la teoría de la transformada de Johnson-Lindenstrauss para aproximar atención softmax

- **Performer:** la matriz de atención puede calcularse mediante una función kernel definida positiva, esto es, puede hallarse mediante alguna medida de similitud $\mathcal{K}(\mathbf{q}, \mathbf{k}) \geq 0$ entre $\mathbf{q} \in \mathbb{R}^D$ y $\mathbf{k} \in \mathbb{R}^D$

► Kernel Gaussiano:

$$\mathcal{K}_{\text{gauss}}(\mathbf{q}, \mathbf{k}) = \exp \left(-\frac{1}{2\sigma^2} \|\mathbf{q} - \mathbf{k}\|_2^2 \right) \quad (76)$$

► Cálculo de la matriz de atención con el kernel Gaussiano:

$$A_{ij} = \exp \left[\frac{\mathbf{q}_i^t \mathbf{k}_j}{\sqrt{D}} \right] = \exp \left[\frac{-\|\mathbf{q}_i - \mathbf{k}_j\|_2^2}{2\sqrt{D}} \right] \exp \left[\frac{\|\mathbf{q}_i\|_2^2}{2\sqrt{D}} \right] \exp \left[\frac{\|\mathbf{k}_j\|_2^2}{2\sqrt{D}} \right] \quad (77)$$

donde el primer factor es $\mathcal{K}_{\text{gauss}}(\mathbf{q}_i D^{-1/4}, \mathbf{k}_j D^{-1/4})$ con $\sigma = 1$ y el resto son solo factores de escalado independientes

- ▷ En general, el kernel Gaussiano puede expresarse como:

$$\mathcal{K}_{\text{gauss}}(\mathbf{x}, \mathbf{y}) = \mathbb{E}[\boldsymbol{\eta}(\mathbf{x})^t \boldsymbol{\eta}(\mathbf{y})] \quad (78)$$

donde $\boldsymbol{\eta}(\mathbf{x}) \in \mathbb{R}^M$ es un vector de características aleatorio derivado de $\mathbf{x}$, basado en funciones trigonométricas o exponenciales

- ▷ Atención softmax en términos de kernel Gaussiano:

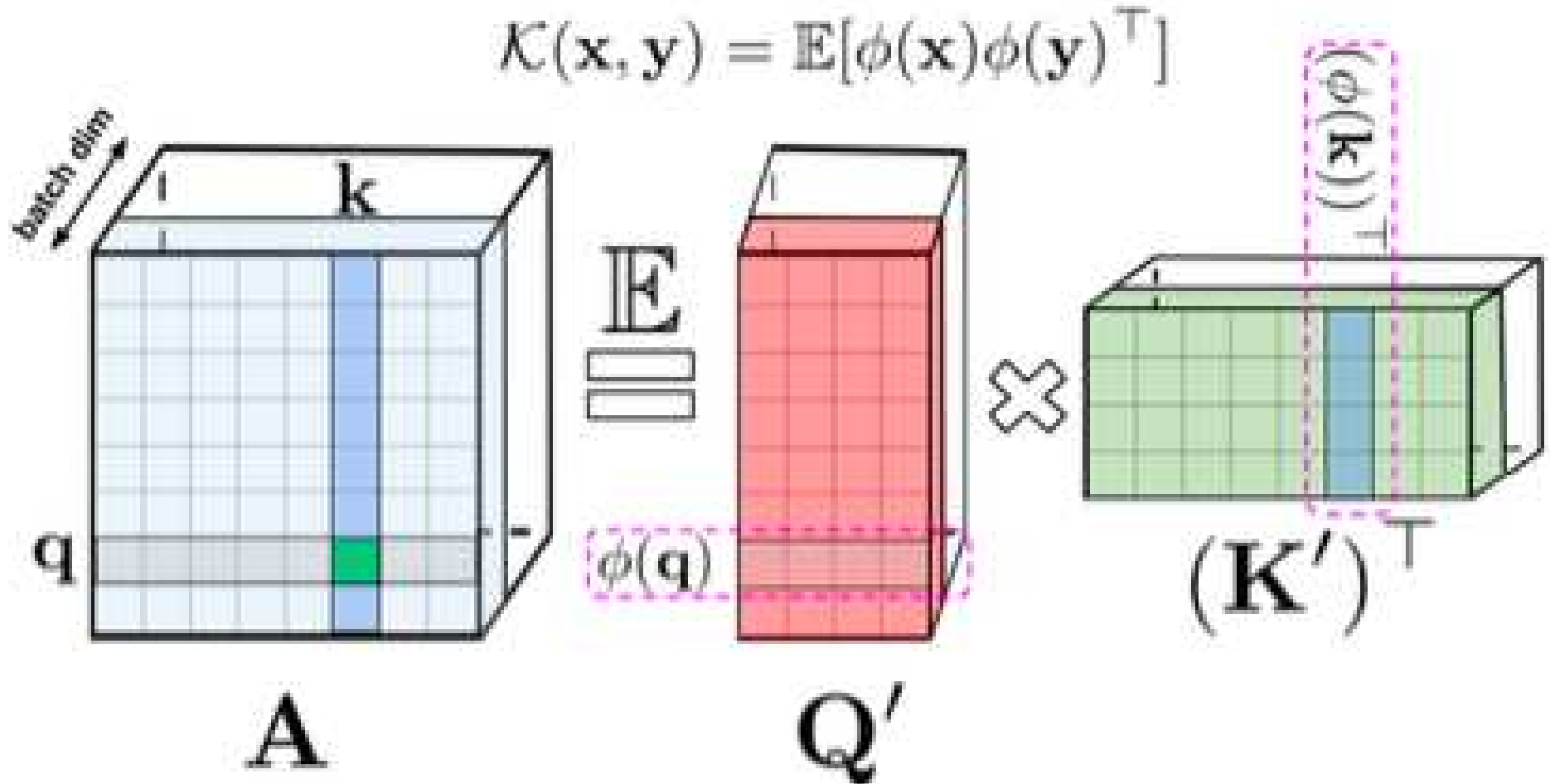
$$A_{ij} = \mathbb{E}[\boldsymbol{\phi}(\mathbf{q}_i)^t \boldsymbol{\phi}(\mathbf{k}_j)] \quad (79)$$

con

$$\boldsymbol{\phi}(\mathbf{x}) \triangleq \exp\left(\frac{\|\mathbf{x}\|_2^2}{2\sqrt{D}}\right) \boldsymbol{\eta}\left(\frac{\mathbf{x}}{D^{\frac{1}{4}}}\right) \quad (80)$$

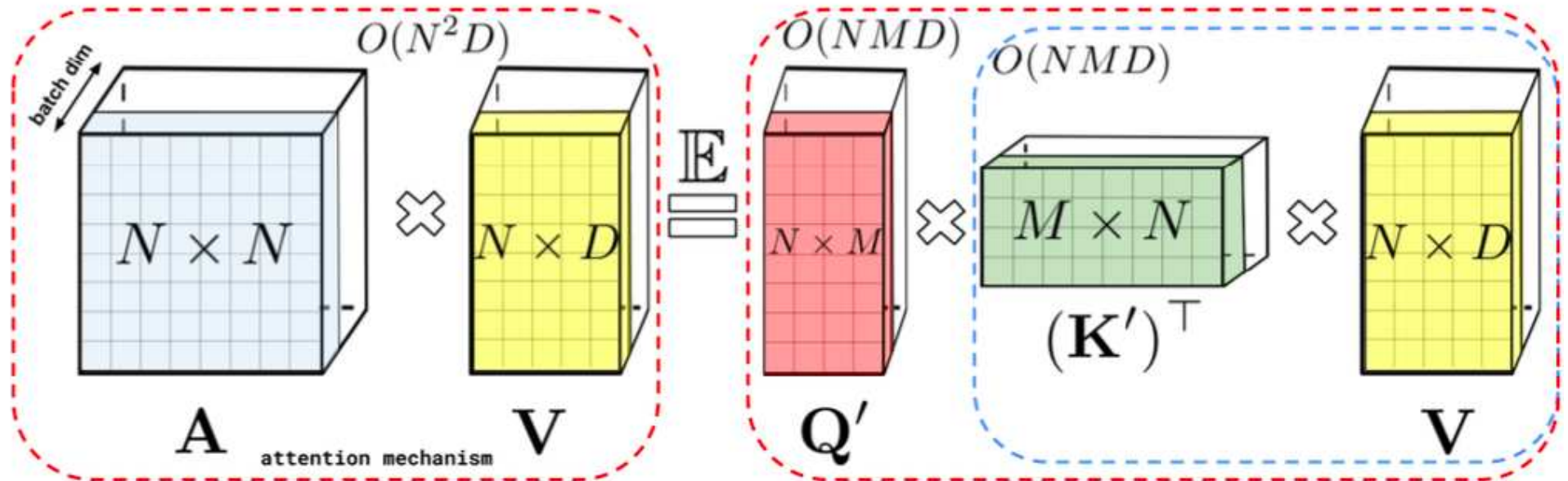
- Matriz de atención completa: queries y claves en $Q', K' \in \mathbb{R}^{N \times M}$

$$A = \mathbb{E}[Q'(K')^t] \quad (81)$$



- Aproximación a $\mathbf{A}$ con M pequeño, por ejemplo $M \in O(D \log(D))$:

$$\widehat{\text{attention}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{diag}^{-1}(\mathbf{Q}'((\mathbf{K}')^t \mathbf{1}_N))(\mathbf{Q}'((\mathbf{K}')^t \mathbf{V})) \quad (82)$$



15.7. Modelos de lenguaje y aprendizaje de representación no supervisado

► *Contextual word embeddings:*

- Las RNNs y los transformers auto-regresivos (solo-decoder) son modelos de lenguaje que generan secuencias de la forma:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid \mathbf{x}_{1:t-1}) \quad (83)$$

- En lugar de un vector one-hot $\mathbf{x}_t$ o un embedding aprendido, usamos los estados latentes de dichos modelos a modo de embedding contextual para tareas seq2seq o clasificación de texto

► *Transfer learning:*

- ▷ *Pre-training:* pre-entrenamos un modelo de lenguaje sin supervisión, con un corpus de texto grande
- ▷ *Fine-tuning:* ajustamos el modelo pre-entrenado a la tarea con un conjunto pequeño y supervisado

► *Modelos causales y no-causales:*

- ▷ Los causales generan texto
- ▷ Los no-causales no generan texto; se orientan a la representación de frases con h_t dependiente de pasado $y_{1:t-1}$, presente y_t y futuro $y_{t+1:T}$

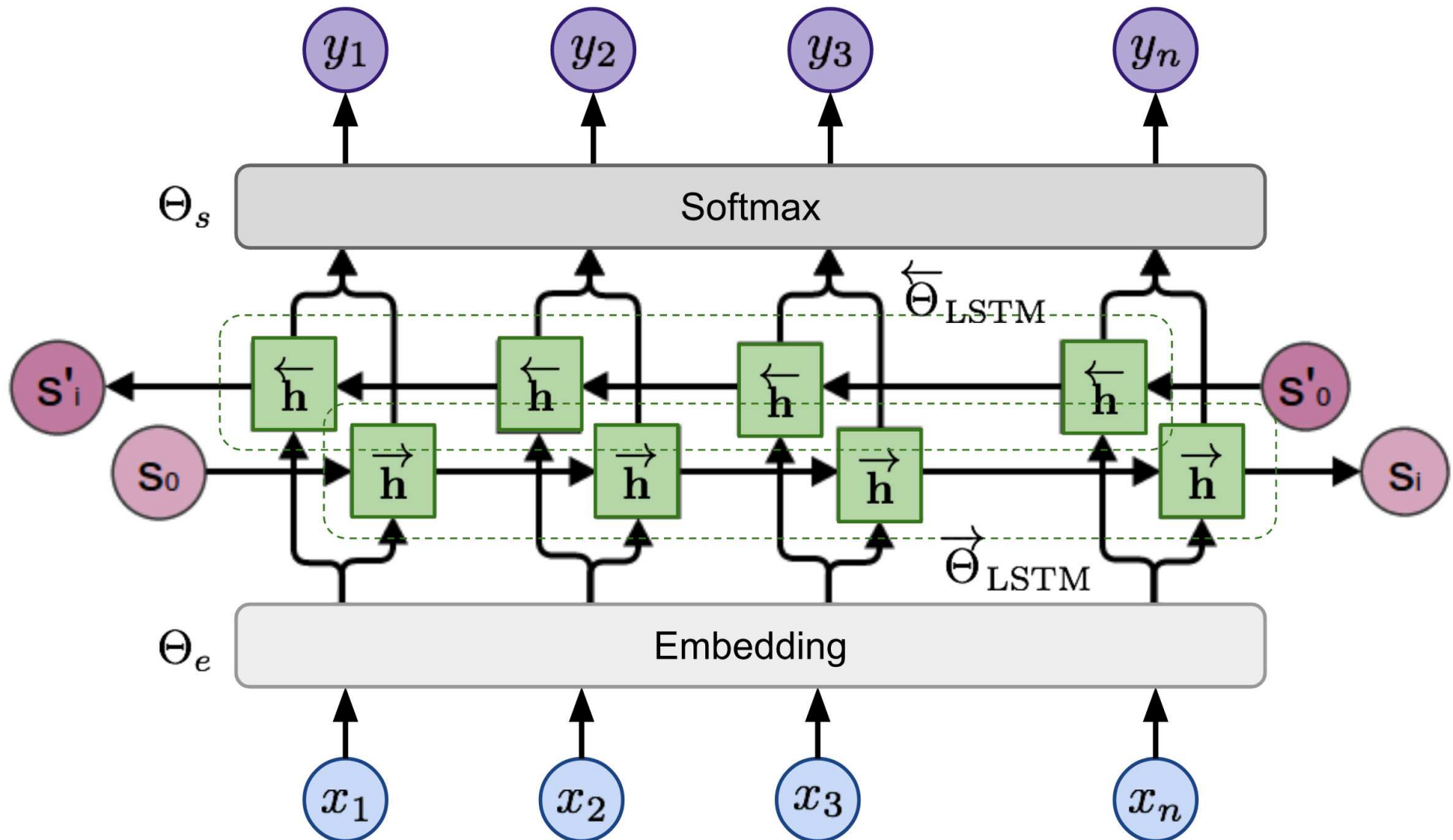
15.7.1. ELMo

- *Embeddings from Language Model (ELMo)*: ajusta dos modelos de lenguaje RNN, uno izqda-dcha y otro dcha-izqda, y combina sus estados ocultos para formar un embedding por palabra
- A diferencia del entrenamiento de una LSTM-BLSTM, con pares entrada-salida, ELMo se entrena sin supervisión, para minimizar la log-verosimilitud negativa de la frase de entrada:

$$\mathcal{L}(\boldsymbol{\theta}) = - \sum_{t=1}^T [\log p(x_t | \mathbf{x}_{1:t-1}; \boldsymbol{\theta}_e, \boldsymbol{\theta}^{\rightarrow}, \boldsymbol{\theta}_s) + \log p(x_t | \mathbf{x}_{t+1:T}; \boldsymbol{\theta}_e, \boldsymbol{\theta}^{\leftarrow}, \boldsymbol{\theta}_s)] \quad (84)$$

con parámetros compartidos de la capa de embedding $\boldsymbol{\theta}_e$, parámetros compartidos de la capa de salida softmax $\boldsymbol{\theta}_s$, y parámetros propios de ambos modelos $\boldsymbol{\theta}^{\rightarrow}$ y $\boldsymbol{\theta}^{\leftarrow}$

- Modelo bidireccional ELMo: $y_t = x_{t+1}$ en modo LSTM hacia adelante y $y_t = x_{t-1}$ en modo LSTM hacia atrás



- ▶ Representación contextual tras entrenamiento: $r_t = [e_t, \mathbf{h}_{t,1:L}^{\rightarrow}, \mathbf{h}_{t,1:L}^{\leftarrow}]$, donde L es el número de capas de la LSTM
- ▶ ***Fine-tuning***: se aprende un conjunto lineal de pesos dependiente de la tarea para transformar r_t en el embedding final de cada token, $r_t^j = r_t^t w^j$, donde j es el identificador de la tarea
- ▶ ***Part-of-speech (POS) tagging***: para etiquetar cada token como nombre, verbo, adjetivo, etc. el modelo aprenderá a dar más peso a las capas inferiores
- ▶ ***Word sense disambiguation (WSD)***: el modelo aprenderá a dar más peso a las capas superiores
- ▶ En ambas tareas se necesita un conjunto pequeño de datos etiquetados específicos de la tarea para aprender un único vector de pesos para transformar $r_{1:T}$ en las etiquetas objetivo $y_{1:T}$

15.7.2. BERT

- ▶ *Bidirectional Encoder Representations from Transformers* o **BERT**: modelo no-causal para representar texto (como ELMo), no generativo, con transformer para transformar una versión modificada de una secuencia de vuelta a su versión no modificada
- ▶ *Tarea fill-in-the-blank o cloze*: la versión modificada omite una palabra o más (texto o palabras cloze) y hay que predecirlas

15.7.2.1. Tarea del modelo de lenguaje enmascarado

- *Ejemplo:* si entrenamos el modelo con transcripciones de vídeos de cocina, podemos crear una frase de entrenamiento de la forma

Let's make [MASK] chicken! [SEP] It [MASK] great
with orange sauce.

donde [SEP] es un token separador insertado entre frases

- Las etiquetas de salida deseadas para las palabras enmascaradas (cloze) son “some” y “tastes”

► *Objetivo:*

- Minimización del negativo de la log-pseudo-verosimilitud

$$\mathcal{L} = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{m}} \sum_{i \in \mathbf{m}} -\log p(x_i \mid \mathbf{x}_{-\mathbf{m}}) \quad (85)$$

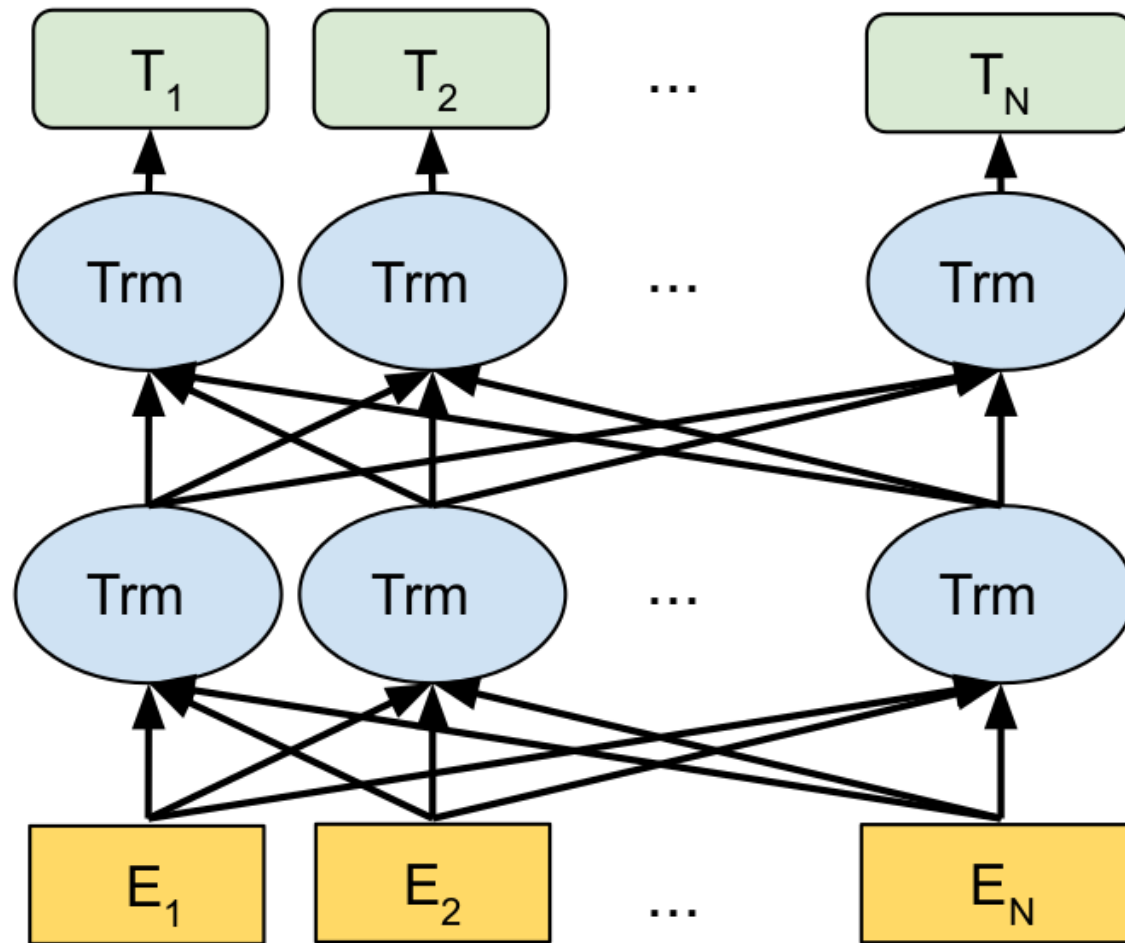
donde $\mathbf{m}$ es una máscara binaria aleatoria

- La pérdida en la posición enmascarada i se basa en la probabilidad condicional dada por una softmax aplicada al vector oculto de la capa final

$$p(x_i \mid \hat{\mathbf{x}}) = \frac{\exp(\mathbf{h}(\hat{\mathbf{x}})_i^t \mathbf{e}(x_i))}{\sum_{x'} \exp(\mathbf{h}(\hat{\mathbf{x}})_i^t \mathbf{e}(x'))} \quad (86)$$

con frase enmascarada $\hat{\mathbf{x}} = \mathbf{x}_{-\mathbf{m}}$ y embedding de x $\mathbf{e}(x)$

BERT



15.7.2.2. Tarea de predicción de la siguiente frase

- *Next sentence prediction task*: un objetivo adicional del BERT original consiste en clasificar si una frase sigue a otra

CLS $A_1 A_2; \dots A_m$; SEP $B_1 B_2; \dots; B_n$ SEP

con token separador especial SEP y token especial CLS para indicar la clase; $y = 1$ si la frase B sigue a la A, $y = 0$ si no

- Aplicaciones: tareas de clasificación de pares de frases, como inferencia en lenguaje natural o similitud textual semántica

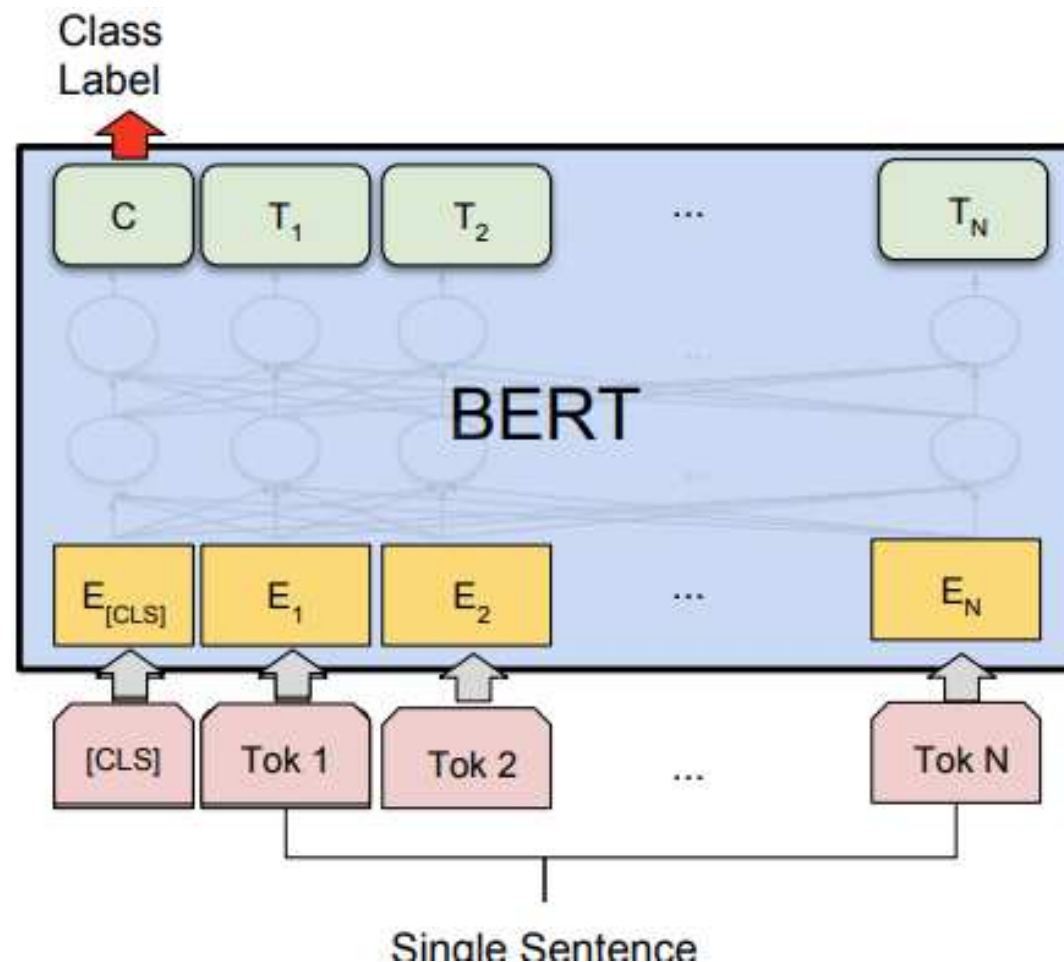
► **Entrada:** codificación basada en la suma de 3 embeddings

- ▷ Uno por token
- ▷ Uno por cada etiqueta de segmento
- ▷ Uno por posición

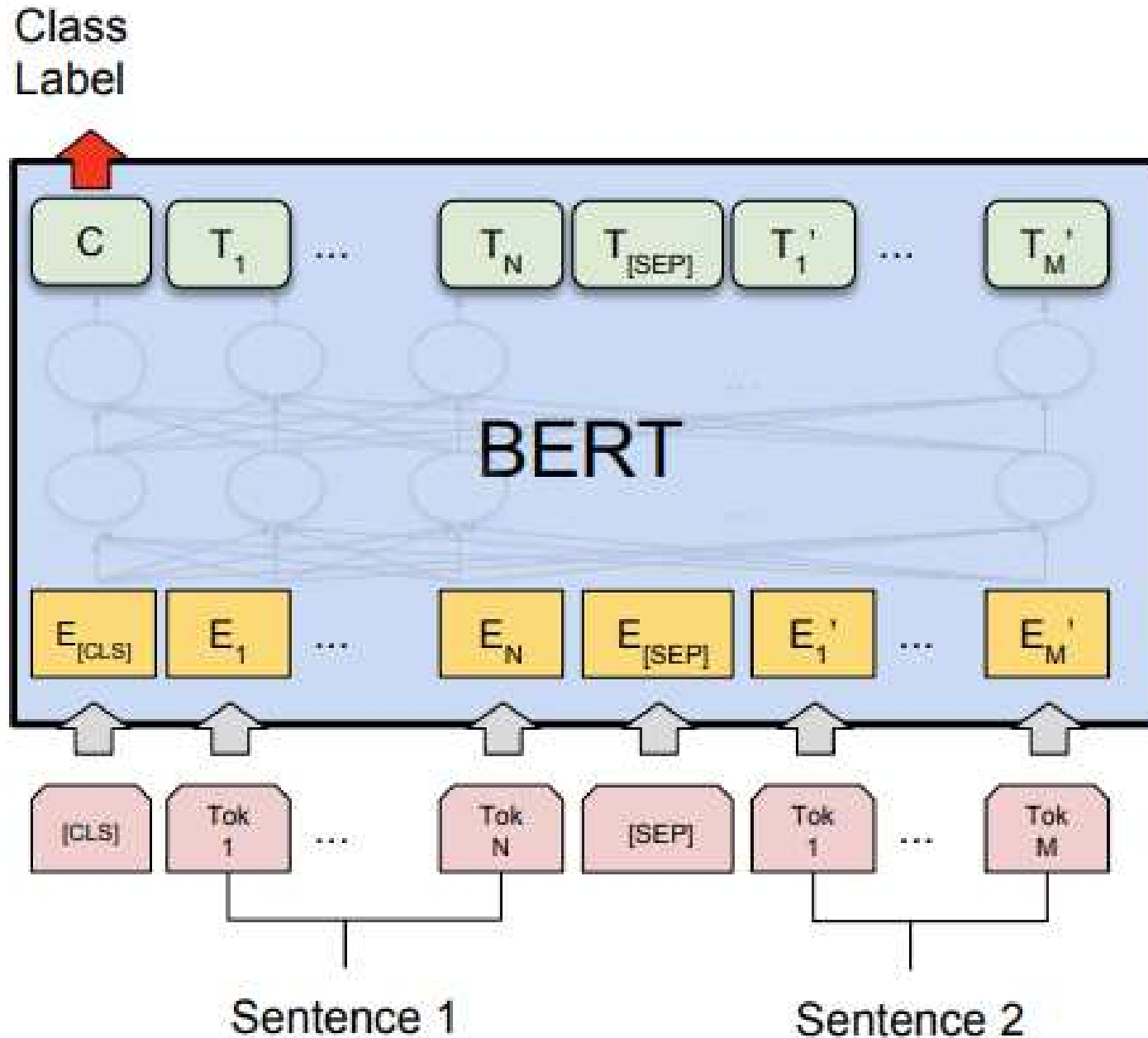
<i>Input</i>	<cls>	this	movie	is	great	<sep>	i	like	it	<sep>
<i>Token Embeddings</i>	$e_{<cls>}$	e_{this}	e_{movie}	e_{is}	e_{great}	$e_{<sep>}$	e_i	e_{like}	e_{it}	$e_{<sep>}$
	+	+	+	+	+	+	+	+	+	+
<i>Segment Embeddings</i>	e_A	e_A	e_A	e_A	e_A	e_A	e_B	e_B	e_B	e_B
	+	+	+	+	+	+	+	+	+	+
<i>Positional Embeddings</i>	e_0	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	e_9

15.7.2.3. Fine-tuning BERT para aplicaciones NLP

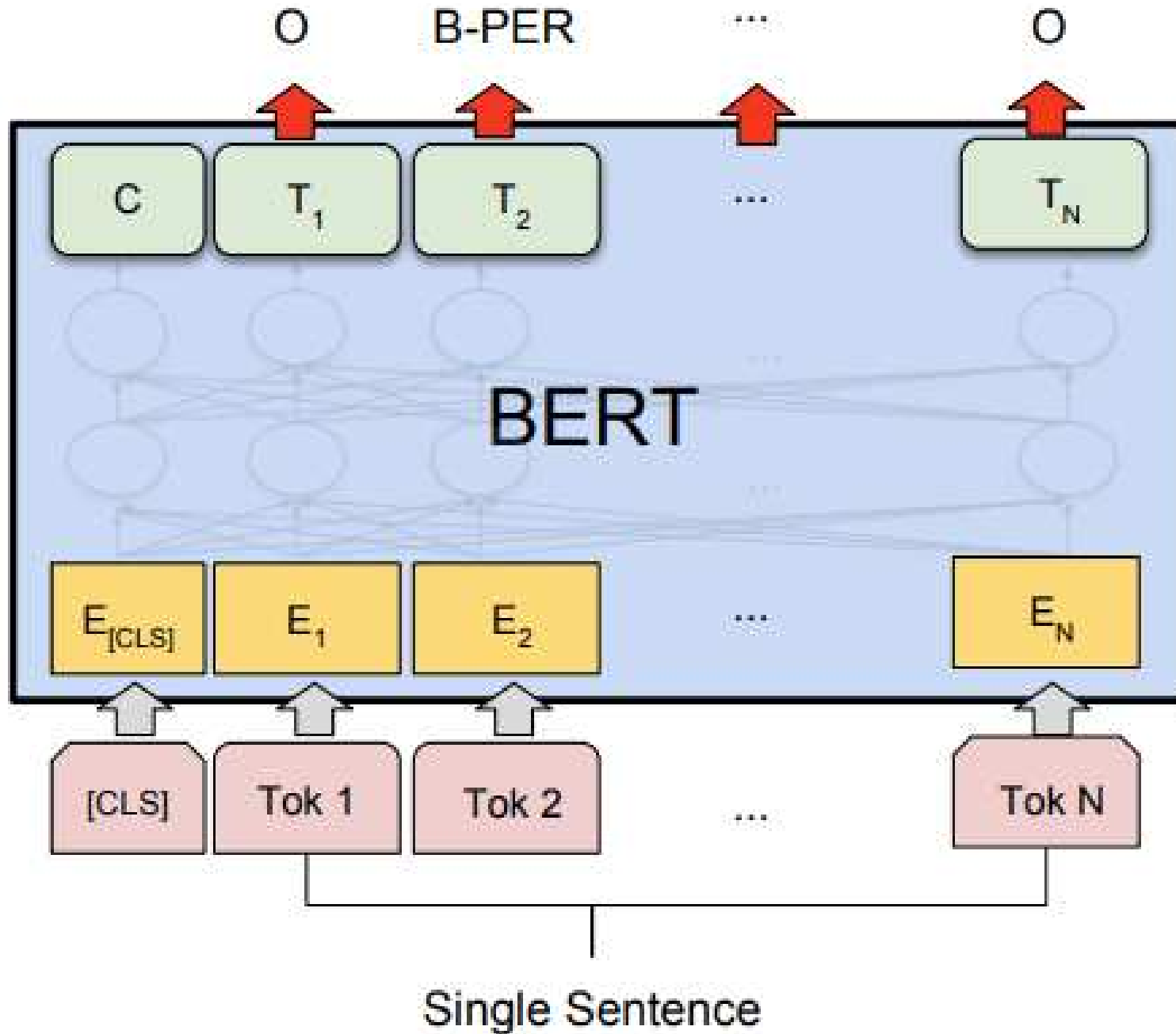
- ***Fine-tuning BERT:*** se añade uno o más cabezales de salida a la capa oculta final para adaptarlo a la tarea deseada
- ***Clasificación de una única frase:*** p.e. análisis de sentimiento, con un MLP alimentado por el vector de características de CLS



- **Clasificación de un par de frases:** p.e. inferencia en lenguaje natural, a partir de CLS con frases de entrada separadas por SEP



- **Etiquetado de frases:** se asocia una etiqueta con cada palabra



► **Part-of-speech tagging:** desambiguación léxica; es una aplicación usual del etiquetado de frases

► **Noun phrase chunking** o **shallow parsing:** análisis sintáctico superficial; identifica entidades de frase como grupos nominales

▷ **Notación Beginning-Inside-Outside (BIO):** ejemplo

1	B	I	O	O	O		B	I		O	B	I	I
2	British	Airways	rose	after	announcing	its	withdrawal	from	the	UAI	deal		

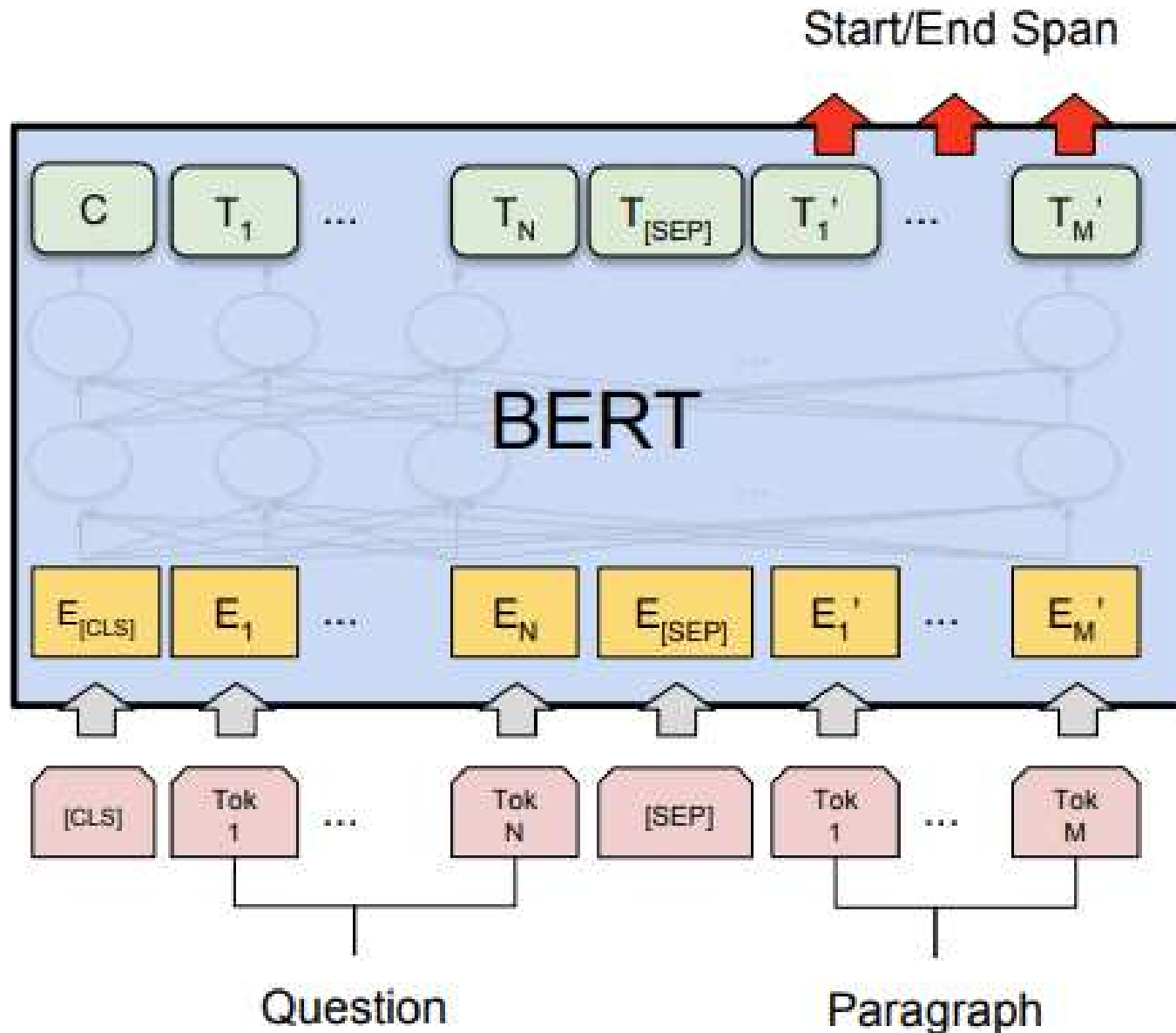
Grupos: “British Airways”, “its withdrawal” y “the UAI deal”

▷ **Named entity recognition:** se reconocen entidades nombradas con su tipo; por ejemplo persona, organización y localización con etiquetas {B-Per, I-Per, B-Loc, I-Loc, B-Org, I-Org, Outside}

1	BP	IP	O	O	O	BL	IL	BP	O	O	O	O	
2	Mrs	Green	spoke	today	in	New	York.	Green	chairs	the	finance	committee.	

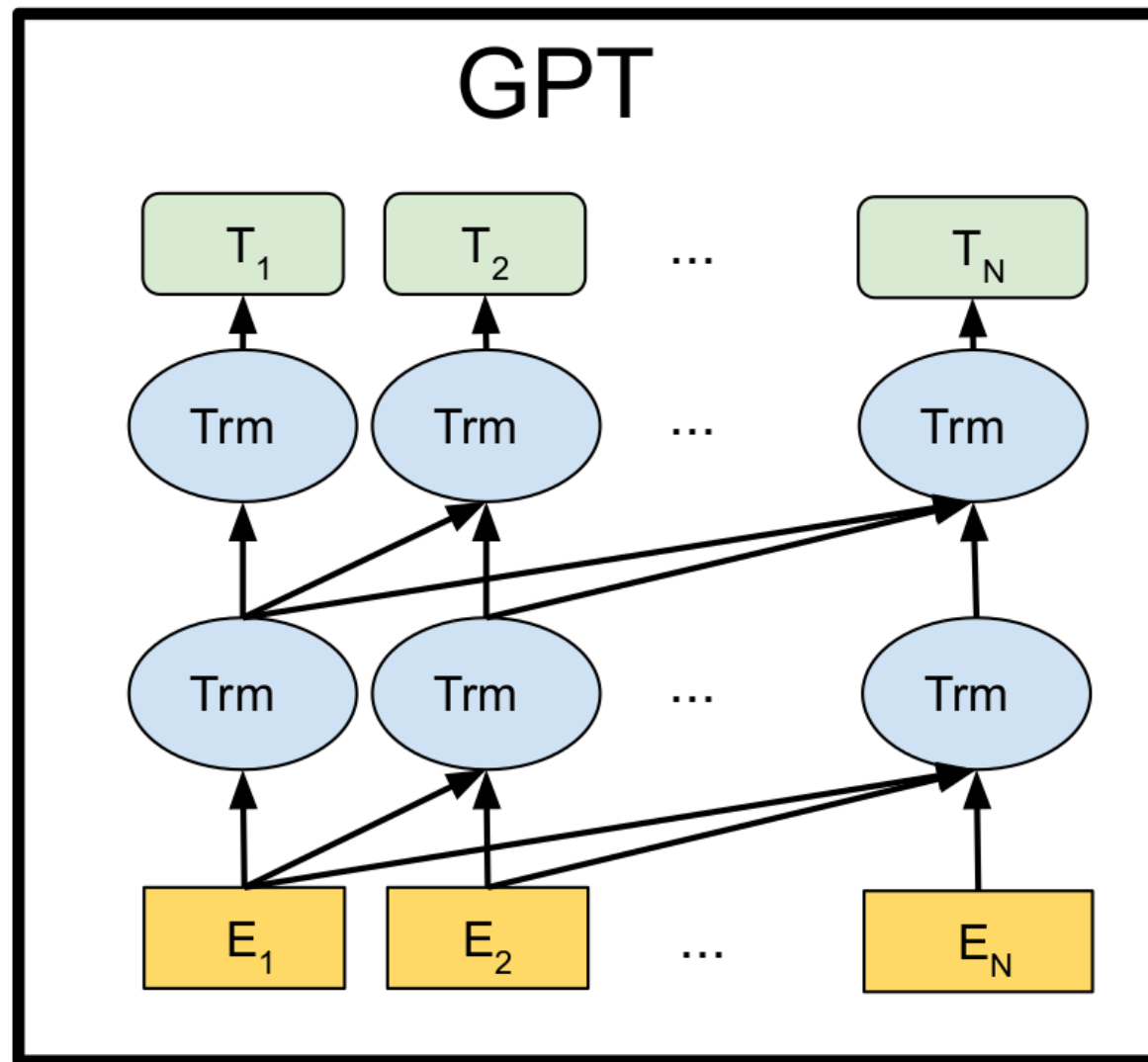
Entidades: “Mrs Green”, persona, y “New York”, localización en la primera frase; “Green”, persona, en la segunda

- **Question answering:** búsqueda de respuestas; dada una pregunta y un texto base, se obtienen las posiciones inicial y final de la parte relevante del texto base con la respuesta



15.7.3. GPT

- *Generative Pre-training Transformer (GPT)*: modelo causal que usa un transformer enmascarado como decoder



- ▶ Objetivo: optimización conjunta sobre un conjunto no supervisado grande y otro supervisado pequeño
- ▶ Objetivo para clasificación:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{LM}} \quad (87)$$

$$\mathcal{L}_{\text{cls}} = - \sum_{(\mathbf{x}, y) \in \mathcal{D}_L} \log p(y \mid \mathbf{x}) \quad (88)$$

$$\mathcal{L}_{\text{LM}} = - \sum_{\mathbf{x} \in \mathcal{D}_U} \sum_t p(x_t \mid \mathbf{x}_{1:t-1}) \quad (89)$$

- ▶ Objetivo de GPT-2: solo el modelo de lenguaje
- ▶ GPT-3: GPT-2 más grande
- ▶ GPT-NeoX-20B: "último" modelo EleutherAI (9-Feb-2022)

15.7.3.1. Aplicaciones de GPT

- ▶ *Zero-shot task transfer*: GPT genera texto a partir de un *prompt* de entrada inicial que especifica la tarea; sin más
- ▶ *(Zero-shot) abstractive summarization*: resumen abstractivo de una entrada $x_{1:T}$; muestreamos

$$p(\mathbf{x}_{T+1:T+100} \mid [\mathbf{x}_{1:T}; \text{TL; DR}]) \quad (90)$$

donde “TL;DR” (“too long; didn’t read”) es un token especial, añadido al final del texto de entrada

15.7.4. T5

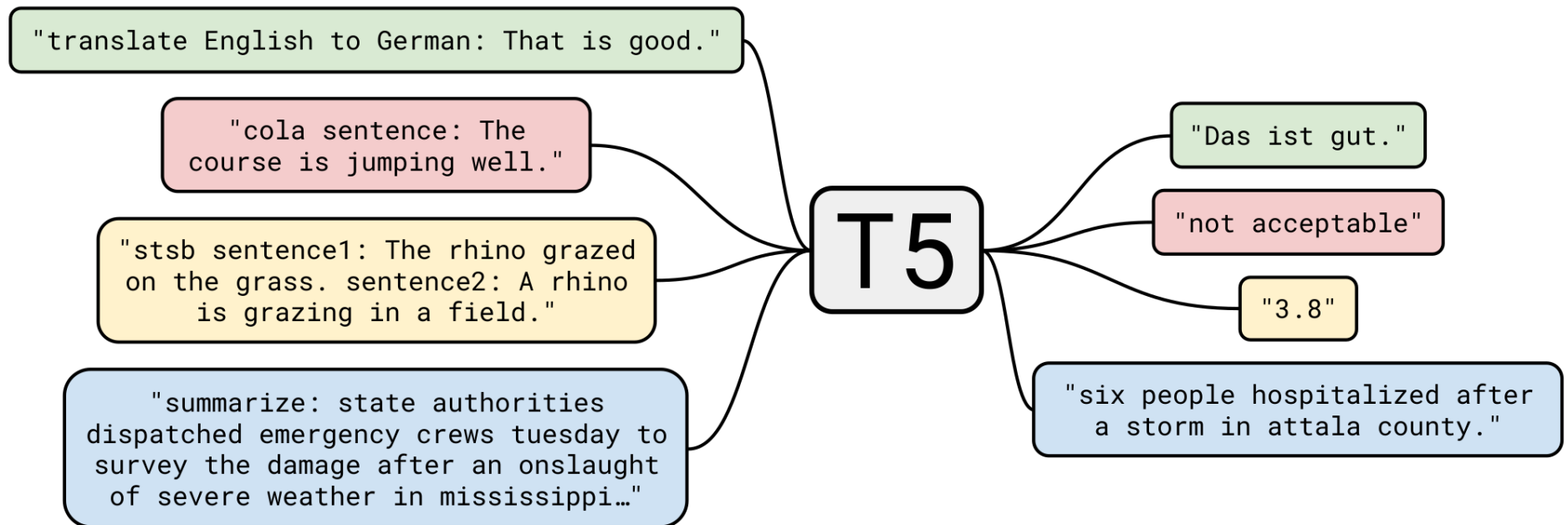
- ▶ **Text-to-text Transfer Transformer (T5):** transformer seq2seq
 - ▷ Pre-entrenado con pares no supervisados (x', x'') ; x' es una versión enmascarada de x y x'' son los tokens perdidos a predecir
 - ▷ Fine-tuned con múltiples pares supervisados (x, y)
- ▶ **Colossal Clean Crawled Corpus (C4):** 750Gb de texto web para entrenar T5 con un objetivo de denoising tipo BERT
- ▶ Ejemplo: frase x , entrada x' y salida (objetivo) x''

- 1 Thank you for inviting me to your party last week
 - 2 Thank you <X> me to your party <Y> week
 - 3 <X> for inviting <Y> last <EOS>

donde <X> y <Y> son tokens únicos del ejemplo.
- ▶ Los pares supervisados se obtienen manualmente

► Tareas:

- ▷ Traducción de inglés a alemán
- ▷ Corpus of Linguistic Acceptability (CoLA): validez lingüística
- ▷ Semantic Textual Similarity Benchmark (STBS): grado de similitud semántica
- ▷ Resumen abstractivo



15.7.5. Discusión

- ▶ *Gran popularidad:* los modelos de lenguaje gigantes como BERT y GPT-3 han generado gran interés
- ▶ *Debate:* “entienden” el (significado del) lenguaje?